

RideSkill：面向广义拼车的分层算法 与 LLM 驱动自动演化

齐健 Zhao1, 2, 夏良 Tong2*, 森 Li1, 3 ??, 明 Yuan2

1 香港科技大学 2 Noah的方舟实验室 华伟 3 香港科技大学(广州)

摘要

拼车(Ride-share),允许不同原产地-目的地对(OD)的多个乘客共用单车,是一个具有挑战性的运营问题,因为它要求不同OD对的订单被高效捆绑,并在不确定和不同的情况下分配给车辆.虽然多代理强化学习(MARL)解决方案取得了有希望的性能,但是,它们受到一般化(适应不同的环境情景)有限,可转移性低(适应不同的平台目标),以及大规模系统的培训困难,如维度的诅咒.最近,在大型语言模型(LLM)规模的推动下,有几部作品将LLM纳入ride-hailing系统,要么直接使用LLM作为决策代理,要么将其用于自动算法设计.然而,这些方法都没有支持车辆共享,这使得问题复杂化,因为状态和动作空间都成指数扩大.此外,它们大多数需要经常的LLM在推断时间打电话,使其无法实时部署.为了解决这些问题,我们提议RideSkill采用LLM辅助自动算法设计的ride-sharing分级方法. RideSkill由一个联合体组成,从学习的技能仓库中为每辆车分配适当的技能,能够在不同的情景和目标下进行适应性调度,以及一个按顺序将闲置的车辆迁移到新兴区域,避免车辆之间的冲突的重定位器.关键是,技能存储器,组合器和重置器都通过基于LLM的自动演化方法进行训练,从而在部署时不再需要LLM通话,从而确保高实时性能.在现实世界ride-hailing数据集上评价,RideSkill与基于模型,MARL和LLM的基准相比,实现了最佳性能.最关键的是,我们的方法显示了不同机队规模、车辆速度、客运能力、订单密度和目标的强烈概括和灵活性,这表明了高实用价值.由于公司政策,我们感到遗憾的是,我们不能发布我们的代码。

1 引言

拼车系统允许多个不同来源的乘客——目的地对(OD)共用单一车辆,已成为城市交通的变革性范例.通过增加车辆占用量和减少出行总数,可以缓解交通拥堵,降低碳排放,提高交通网络的整体效率.除了这些环境效益外,ride-sharing通过降低票价和缩短等候时间来增强乘客的便利性,同时使平台能够以有限的车队满足更多的需求,从而在人口稠密的城市中产生巨大的社会和经济价值(Jin等人,2018年).

——*项目铅:西莲塘 =====

在ride-sharing中发出指令的现有方法基本上分为两类。经典优化和基于规则的方法虽然在计算上是有效的,但典型的偏执性(Alonso-Mora等人,2017a)。多剂强化学习(MARL)技术通过学习长期政策表现出更强的性能,然而当车辆和订单数量增加时,它们却受到维度诅咒(Hao& Varakantham, 2022年)。更关键的是,一旦制定模型或培训政策,它们就难以适应不断变化的环境条件(e.g.、不同的车队规模、车辆速度或能力)或改变平台目标。在实践中,一个平台的目标(例如,最大化服务率,尽量减少乘客绕行,或平衡车辆收入)可能会随着市场条件和政府监管而迅速演变,使得静态解决方案不足。更为严重的是,大多数转移RL方法在多代理环境中都失败了,因为其他代理商政策的变化改变了任何特定代理商经历的环境(Barreto等人,2018年)。

最近,大语言模型(LLM)被引入ride-hailing系统,要么将一个LLM作为直接选择单个车辆订单的决策代理人,要么将其作为全球火柴(Lyu等人,2026a;Zhang&Xiao,2026;Su等人,2025年)。虽然这些方法利用了LLM的推理能力,但它们在推论时需要频繁的模型调用,因此不适合实时ride-sharing的严格潜伏性要求。替代工作线使用LLM进行自动算法发现:通过将LLM的基因能力与进化搜索相结合,可以设计出新的休眠法或政策(Zhang等人,2026年)。与传统的演化算法相比,LLMS注入了丰富的前科知识,并能够更有效地探索算法空间,然而,基于LLM的现有方法是专门为ride-hailing开发的。将它们扩展至ride-sharing是非三角性的,因为它需要考虑可行的订单组合,从而扩大状态和行动空间,模拟顺序之间的依赖性,并处理一旦载客就变得多样化的车辆。更为重要的是,所有这些方法仍然按照常规的MARL解决方案进行单一的固定情景和目标的培训。因此,所获得的政策对不同操作条件的概括性有限,在平台的目标变化时可转让性有限。(附录A提供了详细的文献审查。)

为了应对这些挑战,我们提议RideSkill,一个分级框架,在各种情景和平台目标下进行适应性订单的调度和车辆的重定位。RideSkill保持(一)可重复使用的基本技能仓库(e.g.最近的匹配,绕行最小化,服务率最大化);(二)以当前环境和客观为条件的组合器,动态地为每辆车辆分配适当的技能;和(三)将闲置车辆迁移到新出现的需求区域,同时避免因同时作出决定而产生的冲突的顺序调整器。所有三个组成部分都通过精心设计的LLM辅助演化程序进行脱线培训,其中包含自我检查、多样的任务生成以及群体关系fitness的制定。培训后,由此产生的政策运作时没有任何LLM的呼声,从而满足了实时制约,同时保持强烈的概括性和可转移性。我们用一个用纽约市真实旅行数据构建的大规模ride-sharing模拟器来验证ride-sharing。结果显示,RideSkill超越了不同情景和目标的所有类型的基线。据我们所知,RideSkill是LLM任务的第一个ride-sharing解决方案,也是第一个能够处理不同情景和目标的通用方法。

2 问题定义

本文考虑一个中心化的拼车平台,它运营一支车队。乘客可以在任意时刻向平台提交请求,平台每隔 Δt 个时间单位处理一次请求,即相邻决策步之间的时间间隔,以便进行更好的规划。平台需要决定如何将请求(订单)派给车辆,同时考虑新分配订单与在途订单之间的OD和时间关系、车辆与订单之间的空间关系、车辆剩余容量,以及未来订单可能带来的影响。未成功分配的订单会返回订单池,等待后续处理。不过,乘客是有耐心上限的:如果某个订单在给定时间阈值内没有被确认,该订单就会被取消,从而造成平台潜在收入损失和用户流失。

沿用Zhao等人(2026)的设定,我们将该问题建模为多智能体马尔可夫决策过程(MAMDP),记为 $M = \langle n, S, U, P, R, \gamma, O, T \rangle$,其中 n 表示车辆(智能体)数量,

和其余组件分别代表联合状态,联合行动,州-过渡功能,奖励功能,折扣系数,联合观察,以及时间范围.我们按 $i = \{1, n\}$ 和 $j = J_i$ 待发订单来索引车辆.每一辆 i 的局部观察轴由自己的状态(位置,地位,剩余容量,承诺乘客)和全球请求池组成. (详细概念说明见附录B.)

在传统解决方案中, M 被视作固定的,政策或模型的设计或培训也相应.但在实践中, M 随时间而变化.例如,在高峰与非高峰时段,平台的运行规模和订单量可能有所不同,交通条件可能影响车辆速度.此外,随着市场条件的演化(例如,由于平台之间的竞争),目标 R 可能从收入最大化转向服务质量最大化.在本文件中,我们的目标是开发一种灵活的方法,能够适应这些不同的任务(每个任务都以不同的 M 为特征),而不需要政策重建或再培训,这将需要大量的计算成本和时间。

3 方法

3.1 总体框架

我们方法的目标是设计一种适应不同情景和平台目标的订单发送方法,但是,端到端的学习是行不通的,因为将这类条件纳入一个已经巨大的行动,观测空间极大地扩大了搜索空间。为了解决这个问题,我们提出了一个分级解决方案,如图1所示。(一)一,提供训练有素的基本技能仓库,如最近的匹配,尽量缩短绕行时间,最大限度提高服务率,在不同情况下通常可以重复使用。(二)然后,根据当前情景和目标,由联合体为每辆车分配适当的技能.例如,可以给闲置车辆分配最接近匹配的技能,以尽量提高服务率,而多路订单的车辆可获得绕行-最小的技巧,以确保服务质量。(三)最后,一个重定位器按顺序处理每辆闲置车辆,并将其迁移到需要的地方.通过依次运行,重定位者执行一个分步战略(类似于思维链推理),避免了将过多的车辆迁移到同一区域的风险,因为同时进行处理。

为了优化这三部分,我们引入了一个基于四级LLM的自动设计演化算法,如图2所示,它建立在 $(\mu+++-ES$ (Schwefel, 1981年)上,我们的方法利用LLM来生产儿童,利用他们的缩放能力和一般知识.自我检查模块包含一个自查模块,其中LLM审查并纠正其产出,以确保与自身的政策和客观设计动机保持一致,从而减少进化偏差。为了增强概括性和可转移性,我们设计了一个任务生成机制,将环境随机化与LLM设计的目标结合起来,以创建一套多样化的培训.最后,为了解决不同任务奖励尺度的不同问题,我们提出一种GDPO式(Liu等人, 2026年)fitness的提法,将群体优势概念从强化学习中融入进化算法。

在本节中,我们首先介绍三个组件,并简要说明我们如何使用ES算法来训练它们,然后详细介绍在培训这三个组件时所使用的基于LLM的一般 $(\mu+++-ES$ 框架,详细的即时设计和算法过程载于附录C。

3.2 组成模块

最近,技能存储器的概念,包括明确的文字描述,固定输入和输出定义,以及详细的程序(e.g,代码或函数),显示出LLM的高度潜力,其优点是模块化,可重复使用,可调和,标准化(Zhou等人, 2026年).受这一动机的启发,我们维持着一个ride-sharing技能存储器,可以被驱动器(i.e,代理人)使用或组合.在存储器中,有一系列由LLM探索和演化的基本原子技能,如最近的匹配.技能存储器由 $B = \{s_1, \dots, s_K\}$, 每一个技能 s_k 都有自成一体的订单发送政策,并附有自然语言卡(客观,描述,机制),所以人可以审计它以解释性。

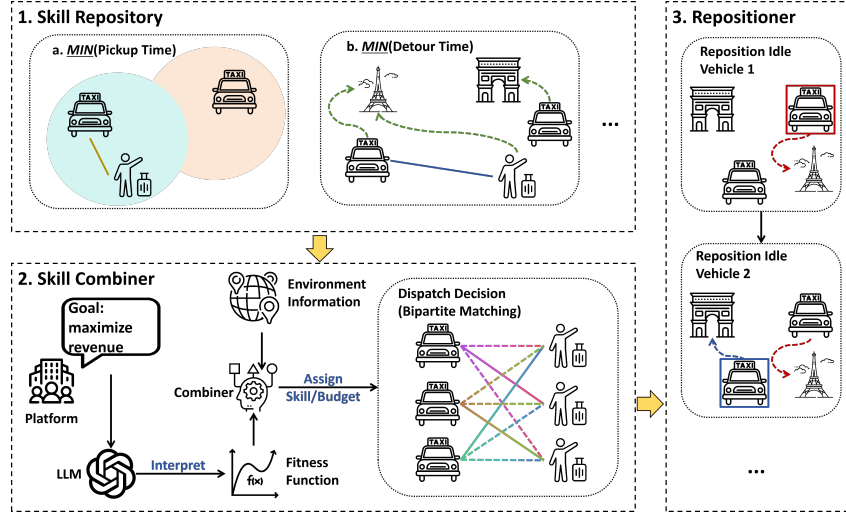


图 1：RideSkill 的组成模块。平台目标和环境上下文由组合器解释，并为每辆车分配技能权重；随后，重定位器将空闲车辆依次调度到新出现的需求区域。

由于ride-sharing调度任务中的联合行动空间很大,因此很难直接输出分配给每个车辆的顺序,在目前的MARL范式中,我们大多首先对每对车辆订单(e.g.,Q值(徐等人,2018年)或匹配概率(Zhao& Li,2025年)进行比对,然后使用双方匹配,以最大化全球得分. 因此,我们认为重要的是分数函数,我们正式确定每种技能的政策,作为对每对车辆订单的分数函数。在形式上,一个技能Sk将一号车的观察 X ,候选订单 oj ,插图-静态上下文 ep (e.g.,公路网,车队规模,区域布局),以及每步活的上下文 $step$ (e.g.,待清点,以及下文引入的每个区域的需求/消耗状态)绘制成该车订单对的实值边值:

$$sk_{xi, oj, ep, step} \rightarrow R. \quad (1)$$

技能产生的每平方分数通过解决双方匹配问题转换成混凝土调度,制定成整数线性程序,将每个顺序分配给最多一个车辆,每个车辆最多每个决策步骤一个新顺序:

$$\max_{\{u_{i,j}\}} \sum_{i \in \mathcal{I}} \sum_{j \in \mathcal{J}_t \cup \{\emptyset\}} u_{i,j} \cdot a_{i,j}, \quad (2a)$$

$$[菲律宾]. \quad u_{i,j} \leq 1, \quad \forall j \in \mathcal{J}_t, \quad (2b)$$

$$\sum_{j \in \mathcal{J}_t \cup \{\emptyset\}} u_{i,j} \geq 1, \quad \forall i \in \mathcal{I}, \quad (2c)$$

$$u_{i,j} = 0, \quad \forall i \in \mathcal{I}, \quad j \in \mathcal{J}_t, \quad (2d)$$

$$\sum_{j \in \mathcal{J}_t} u_{i,j} \cdot h_j \leq c_i, \quad \forall i \in \mathcal{I}, \quad (2e)$$

$$u_{i,j} \in \{0, 1\}, \quad \forall i \in \mathcal{I}, \quad \forall j \in \mathcal{J}_t \cup \{\emptyset\}, \quad (2f)$$

在离线培训期间,每种技能完全由LLM编写和培训,具体地说,要求LLM自己探索各种技能,其中每种技能应具有不同的目标,而且

在每次迭代中,LLM都提出新的技能,连同其目标、机制以及一个fitness功能,而不是推出的度量标准。该提议经过编译、沙盒验证,由基于LLM的(QQ)-ES算法演化而成。一旦技能训练有素,其评价衡量标准就会与现有的衡量标准进行比较(通过类似度量标准,如余弦相似性),以防止近似重复的积累。当寄存器达到K技能或Nf连续回合未能产生新的不同行为时,循环就会停止。

组合器与多代理LLM系统一样,不同的车辆有不同的情况,需要相应的适当技能。为了实现这一点,我们引入了一种组合器,向以自身状态、环境条件和目标为条件的代理人分配适当的技能。

$$\pi_C(x_i, \phi_{ep}, \phi_{step}, w) = \alpha_i = (\alpha_{i,1}, \dots, \alpha_{i,K}) \in \mathbb{R}^K, \quad (3)$$

这些分数在发车时间转换为混合weights,方式是保持b 最高阳性技能, $K_i = \text{top-b } \{k : \alpha_{i,k} > 0\}$?? , 并应用软max:

$$p_{i,k} = \frac{\exp(\alpha_{i,k})}{\sum_{k \in K_i} \exp(\alpha_{i,k})}, \quad k \in K_i, \quad (4)$$

因此, 由于目标由用户提供,它可能采取出乎意料的形式,甚至是一个黑匣子,因此很难直接将这样的函数编码为组合器。为了解决这个问题,我们提出了一个事件探测机制,其中LLM会演化出一组基本的探测事件(例如,向配有路由指令的车辆下达命令)与组合器并列;组合符调用这些事件,并读取配对响应之间的差别,以估计奖励函数的形状。在任务生成过程中,每个作者目标的系数常态化为一个(一个保持任期比例的单一全球调整尺度;见第3.3节),因此被调查的响应在目标之间具有可比性,并有不同的奖励尺度。

产生的混合型(PHA)用于给每个待决的订单打分,使每个技能都比车辆候选人的一组技能正常:

$$a_{i,j} = X \left(\frac{t_{step}}{\text{千元}} \right) \frac{\sum_{k \in K_i} \alpha_{i,k} \cdot \phi_{step}(x_i, \phi_{ep}, \phi_{step})}{\sum_{k \in K_i} \alpha_{i,k}}, \quad (5)$$

技术K的分数与能力K的分数之间有平均和标准的偏差。技术K的分数与车辆的分数相比,我可行的候选人设定了 J_i QJt(我们省略了他们对i的依赖,以示简洁),而技术K的分数是0。技术K和K的分数是0。这种技能标准化是必要的,因为不同的技能产生不同范围的分数,而混合这些分数会直接让大范围技能摧毁其他技能的信号。联合体是在训练有素的技能仓库下,通过基于同一LLM的(XQ)-ES的同一LLM-ES,在目标和设想方案的分配上发展探险和路径逻辑。

此外,受每个参与者预算不同的投标机制的启发,我们可选择地调整对应的分数,以促进车辆之间的公平。直觉是,为了使驾驶员的奖励(反映其收入)更加相似,我们应该给那些历史累积奖励较低者更多的预算。从形式上讲,我们计算每个车辆的收入预算:

$$\beta_i = \exp(-\rho \cdot z_i), \quad z_i = \frac{E_i - \bar{E}}{\sigma_E}, \quad (6)$$

E_i 是i的累计带回家收入,“E”和“ $\bar{E}$ ”是整个车队的累计收入的平均值和标准差,“0”是公平强度参数,“0”是小常数。低于平均水平的车辆($z_i < 0$)得到“ $\beta_i > 1$ ”,这提高了它们在“Eq”(2)中的对应分数,而高于平均水平的车辆被挡住,“ $\beta_i = 0$ ”是机制完全失效。

请注意,这种缩放机制具有弹性,没有理论保证,从经验上讲,我们观察到这种机制是有效的,不会对整体性能产生实质性的影响。此外,这种机制是可选的,可以由用户自行决定启用或禁用。

需求和闲置车辆供应在空间和时间上的分布可能不均匀,因此,在联合调度订单发出后,许多车辆最终在需求低的地区闲置,浪费了能够满足近未来需求的能力。因此,我们引入了一个重定位器,主动地将闲置的车辆迁移到最有可能搭车的地区,从而改善覆盖范围,并缩短乘客在订单到来之前的等候时间。但是,如果重定位器同时处理所有车辆,那么大型的联合状态和行动空间就很难探索一个良好的策略,并可能导致太多车辆出现在一个单一的地区。为了解决这个问题,我们按随机排列顺序一次处理闲置车辆:每辆车辆被分配到目标区域后,在下一个车辆决定之前,该区域的有效需求就会减少,这自然会把车辆分散到不同地区,并防止同时聚集的车辆供应过多一个热点。在形式上,让 G 表示一组区域,并且 $s = (ob)g = g = G = G$ 表示每个区域共享的有效需求/供给状态,这是 Q 步骤的一个组成部分,但被明确传递给重定位者,因为它在连续行驶时会发生变异。重定位者还采用了车辆的公平预算 β_i ,以便它能偏向于被提升的车辆。每个闲置的车辆,重定位者用一个学习过的功能来评分每个区域,设置 G_i QQG(车辆的当前区域、邻居以及全球最热的区域):

$$\pi_r(x_i, \phi_{ep}, \phi_{step}, \kappa, w, \beta_i) = \{\nu_g\}_{g \in G_i}, \quad (7)$$

g. 车辆被发送到最高评分的 g ?? g ?? G_i ?? G_i ?? g , 之后 o ?? g ?? s 已减速。重定位器像组合器一样,通过相同的四级程序演化。与前两个阶段不同的是,它的fitness被设计为比同一任务和随机种子的重定位基线Grepo - Goff(第3.3节)来改进剧集的回归,而不是原始回归。因此,一个正三角形直接表明,重定位会真正比无所事事地提高性能。

3.3 训练过程

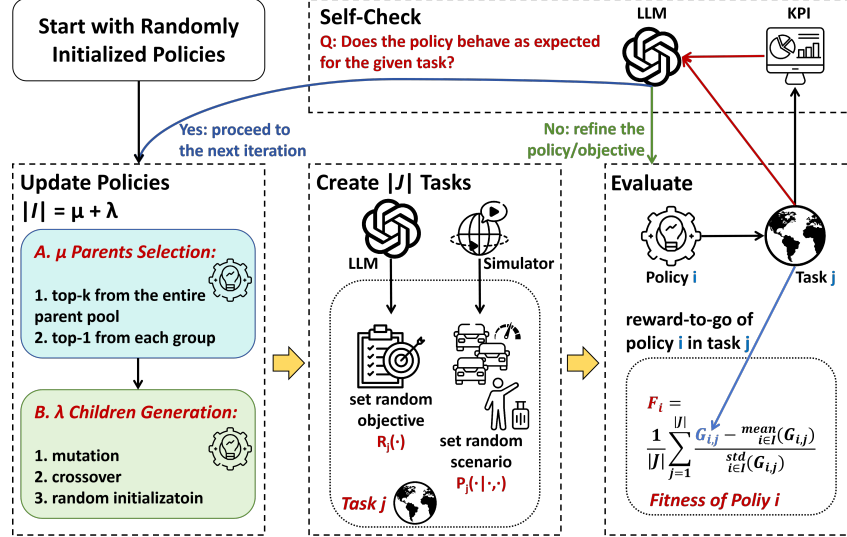


图2: LLM辅助进化训练管道每代人(一)选择 μ 型父母并产生QQ子女,(二)评价每个候选人完成一批任务,计算组别相对优势为fitness,(三)通过自我检查反馈结果,然后进行下一次迭代。

为了训练上述三个组件,我们使用了(??)-ES(Schwefel, 1981年)。最近,LLM在自动算法设计(Zhang等人, 2026年; Su等人, 2025年)中表现出了很高的潜力,他们的高理解度、分析和一般知识可以帮助更高效地演化算法探索,而不是像常规ES解决方案那样随机生成每个迭代的子。以LLM为基础的(XX)-ES的整个过程如图2所示,我们在图2中仔细地调整了我们的任务。

每一代人的程序分为三个阶段。(一)第一,从当代人中选择亲子:由fitness选择顶级-亲子,在第2阶段和第3阶段中,每个任务组(e.g)各保留一名精英(e.g),一个目标家族,或一个由LLM自己定义的公平力量带),这样,从事艰苦工作的专家就能够作为交叉材料生存下来。(二)第二,从这些父母中产生子女: a 双亲的LLM 交叉,并有概率交叉,否则是单亲家庭的突变,加上每代一个无父母的新鲜注射来鼓励品种。与常规ES 不同,这些操作是基于对父母的结果分析,而不是随机探索。每个候选者都要接受沙盒的编译和实地验证;失败会随着温度的逐渐降低而反馈。(三)第三,每个候选政策通过推出一组任务来评价 $T = \{M_m \text{ ?? } T \text{ ?? } m \text{ ?? } 1\}$,其中每个任务 M_m 包含一个平台奖励功能 R_m 和环境动态模型 P_m 。每次搜索最后都进行自我检查,对照原设计意图审查所计量的推出情况。这些阶段的核心设计详细如下。

LLM作为进化操作者。Standard(QQ)-ES使用特定域的突变和交叉操作者(例如,Gaussian 扰动真实值的矢量),我们用LLM来取代这些操作者,因为LLM以前对ride-pooling发送的了解,密码基常规,以及技能/组合合同,使得它能够产生超越系数扰动的结构多样性后代。此外,我们遵循(Su等人,2025年)中的快速设计,让LLM根据父母的表现有针对性地产生孩子。这也使得探索更加多样,因为LLM可以直接编辑功能,而常规方法必须依靠一些预先定义的操作者。

自行提出多种任务。在常规的ES和ride-sharing方法中,任务由设计师预先确定,使得训练有素的算法难以适应分配外的任务。在我们的方法中,LLM提出了它自己训练的各种目标和情景。在第一阶段(技能存储器培训)中,LLM自创了每种技能的目标,并写出了自己的fitness功能,因此,任务批次只包含环境假设。在第二阶段(合并培训)和第三阶段(重定位培训)中,奖励功能来自一些LLM所授权的结构不同的家庭,包括来自自然语言简况的模型作者的目标。具体地说,对于每个目标,情景设置都是随机生成的,以推动经过训练的算法来适应不同的任务,并具有向看不见的任务转移的能力。

不同等级的GDPO式的fitness:由于不同的任务的奖励幅度大不相同,我们采用了受GRPO(Liu等人,2026年)启发的提法,使fitness的任务变化不大。第一阶段则通过自编自导的fitness本身在演化过程中的技能(第3.2节)来评定其候选人的级别,该技能定在0代。

$$AL, m = GL(M) ++(m) \\ (m) \text{ 和 } (8)$$

$\mu(m)$ 和 $k(m)$ 是返回 $\{GL(M_m)\}$ 相对于任务评价的一组候选人的平均值和标准偏差。在第三阶段,返回 $GL(M_m)$ 在 Eq 中。(8) 被配对的改进 $Grepo \text{ } l(M_m) - Goff(M_m)$ 所取代,两种返回在同一个种子上滚动,并重新定位(使用候选人 +l) 而不是关闭,因此,归根结底,重定位除了无所作为之外,还能帮助多少。政策 +l 的 fitness 是对批次中所有任务的平均优势:

$$F_l = \frac{1}{|\mathcal{T}|} \sum_{m=1}^{|\mathcal{T}|} \text{阿尔}, m, (9)$$

而每次搜索最后都会有一个径流,其中不同的圆领队在新批次上重新滚动,以选择最终的冠军(算法2和3),这样我们就可以区分每项任务中政策相对的表现。相反,如果我们简单地使用原始回报的总和,低回报范围任务的信号会被大范围任务消灭。

带有反馈循环的自我检查。使用 LLM 来进行算法设计和演化的一个核心想法是每个设计和过程的高度可解释性。然而,由于知识的偏差

或幻觉,某些拟议的政策或目标可能表现出不同于LLM直观设计动机的性能,使得描述不可信。为了解决这一问题,我们建议在搜索结束后采用自我检查反馈循环。它有两种形式,视阶段而定。在第1阶段,技能审计对照原设计意图对冠军的量子度推出进行审查,并返回三个判断之一:匹配(freeze as-is),描述-错误(技能做一些一致的,但不是卡片声称的;重写卡片),或健身-错误(作者fitness奖励错误的行为;重写fitness和重新研究)。如果审计失败,则过程会回到孩子的产生过程中进行完善。在第2和第3阶段,fitness是固定的,因此,这个审计守护者的失败是完全忽略目标w的程序;政策审计将它衡量为反事实(将同一种子与程序一起滚动,而不交出程序w),并在冻结文物旁边记录客观反应判断,而不反馈到搜索中。

4 实验

模拟和数据.为了验证我们提出的方法,我们在纽约市曼哈顿使用现实世界ride-hailing数据进行了一系列实验(Taxi& Commission, 2024)。模拟运行在RideGym(Zhao等人, 2026)上,一个带有标准化GymAPI的ride-sharing模拟器上。我们使用2026年4月6日至4月12日的8:00至20:00作为训练集,4月13日和4月14日的8:00至20:00作为验证和测试集。

基准.我们比较了不同类型的方法:(一)基于模型的方法:最近的匹配(Kalyanasundaram& Pruhs, 1993年)、Kuhn-Munkres(KM)(Kuhn, 1955年、Simonetto等人, 2019年)和Gale-Shapley(GS)(Gale& Shapley, 1962年、Yue等人, 2024年);(二)基于MARL的方法:REDA(Holder等人, 2025年)、BMG-Q(Hu等人, 2025年)和MF-DDQN(Li等人, 2019年)(表中称为MFRL);(三)基于LLM的方法: Zhang等人(2026年)(包括开放-loop和贴近-loop版本),为了公平起见,我们的方法和他们的方法都采用相同的Claude Opus 4.8模式(Anthropic, 2026年),这里,我们不使用表2中提到的其他方法,因为这些方法很难被推广到现实世界的大规模情景,所有特工都以舰队=1000,容量=4,速度=35km/h的锚定目标接受过训练。

为了评价我们设计的每个单元的效果,我们比较了(一)完整的RideSkill,(二) RideSkill (w/o 代表) 的性能,我们取消了重定位器,以便与基于模型和MARL的方法进行比较更为直接,因为它们没有重定位的设计,(三) 单一技能.我们评价在训练有素的技能仓库中设计的单一技能的平均性能,在部署期间,配对者对每辆车60名候选人的订单(可行的候选人组合 J_i)进行评分,并混合顶级-b=3技能。

实验结果:如表1所示,我们比较了我们的方法在各种假设情况下的基线、报告多个小时的平均结果、车队规模、速度和能力。对于每个配置来说,只有一个设定与基准所用的训练条件不同(即跨小时测试与域内设置相对应,其余的变异是域外设置)。我们的方法始终在所有设想中实现最佳性能,并且几乎在所有度量标准中实现最佳性能。特别是,绕行时间大大低于相互竞争的方法,这表明基于LLM的设计能够有效找到高效的订单捆绑解决方案,我们还注意到,所有基于MARL的基准都未能处理车辆容量超过培训范围的情景,因为它们的固定网络输入大小。相比之下,我们的方法在这些情况中大大超过了基于规则和基于LLM的基线。最后,RideSkill的绩效超过了RideSkill(w/o repo),这证实了我们重定位机制的有效性。这两个变体都大大超过了单技能基线,这表明任何单一战略都无法适应所有情景,证明了我们联合设计的必要性。对不同情景的这种灵活性是我们的方法与先前方法之间的关键区别,由于页数限制,我们将对公平机制的详细分析、客观的调整、与其他有限责任公司的业绩以及经过培训的例子推迟到附录D。

表 1：在给定变量轴的所有取值上计算的均值±标准差。Hour：08–19（12 个取值）；fleet：200–1,500（7 个取值）；speed：20–50 km/h（7 个取值）；capacity ≤4：1–4（4 个取值）；capacity >4：5–8（3 个取值，排除 MARL）。锚定目标；其他变量保持在域内设置。加粗表示最佳结果。

方法奖励服务完成 等待绕道 UTI	
小时	最近的6,051±906 0.903 0.06 0.75±0.06 1.72±0.37 7.00±0.24 1.77±0.30 0.72±0.14 KM 5,765±764 0.88±0.08 0.74±0.08 1.59±0.27 7.08±0.27 1.90±0.34 0.68±0.11 GS 5,480±693 0.85±0.12 0.70±0.12 1.64±0.12 1.64±0.28 7.35±0.58 1.92±0.36 0.67±998 0.92±0.06 0.79±0.08 1.44±0.13 6.60±68 1.15±0.64 0.78±0.14 MFRL 6 850±1 042 0.93±0.06 0.810.08 1.51±0.10 6.53±0.65 1.13±0.61 0.80±0.14 BMG-Q 657±1,0.92±0.080.070.07 0.700.080.080.06. 6.79±0.53 1.38±0.47 0.78±0.14 张等(2026)(公开) 5,780±787 0.88±0.07 0.73±0.07 1.86±0.46 7.12±0.25 1.90±0.31 0.70±0.14 张等. (2026)(接近) 5,835±811 0.88±0.07 0.74±0.07 1.83±0.45 7.06±0.22 1.84±0.28 0.70±0.14 单刀杀人 4,876±970 0.72±0.05 0.61±0.04 2.35±0.08 5.91±0.20 1.12±0.11 RideSkill (w/o reps) 7,221±1,360 0.88±0.07 0.79±0.06 1.14±0.07 5.05±0.27 0.17±0.02 0.80±0.10 RideSkill 7,654±1,497 0.92±0.06 0.83±0.05 1.12±15 5.24±2 0.90±0.10
	最近的5,288±2,801 0.63±0.24 0.52±0.21 2.25±0.30 7.63±0.47 2.66±0.62 0.90±0.08 KM 4,981±0.23 0.50±0.20 1.98±0.22 7.64±0.35 2.69±0.46 0.84±0.10 GS 4,390±2,405 0.50±0.50±0.25 0.40±0.21 2.42±0.71 9.38±1.80 2.60±0.39 0.82±0.10 REDA 5,681±3,245 0.64±0.25 0.53±23 1.73±29 8.06±0.91 2.76±1.02 0.02 0.06 MFRL 5,761±3,3,377 0.65±26 0.54±0.50.24 1.76±27 0.00±0.96 2.73±1.06 0.94±4 BMG-Q 5.76±3, 200 0.64±3 0.64±20 1.62±0.23 8.01±0.84 2.77±1.00 0.92±0.05 张等 (2026年公开) 4,842±2,753 0.59±0.26 0.48±0.22 3.14±1.07 7.60±0.33 2.52±0.34 0.88±0.09 张等. (2026年) (接近)4,963±2,729 0.60±0.25 0.49±0.22 2.91±0.81 7.40±0.18 2.44±0.29 0.88±0.09 单科4,835±1,862 0.49±0.18 0.42±0.16 2.06±0.06 6.16±0.25 1.36±0.16 0.73±0.07 RideSkill(w/o代表) 7,517±2,638 0.63±0.21 0.58±0.19 1.16±0.08 4.14±0.68 0.19±1 0.85±0.08 RideSkill 7,907±2,945 0.66±24 0.61±0.21 1.22±10 4.24±0.800.200.20
舰队	最近的6,701±2,158 0.77±0.14 0.63±0.16 2.41±0.85 7.72±1.96 2.30±0.50 0.89±0.02 KM 6,135±1,710 0.73±0.12 0.59±0.14 1.97±0.50 7.79±1.83 2.41±0.35 0.79±0.02 GS 5,429±1,557 0.63±0.13 0.50±0.14 2.11±0.58 8.84±2.34 2.44±0.30 0.77±2.02 REDA 7 078±2,349 0.78±0.13 0.64±0.16 1.75±0.67 8.02±23 22 22 22 0.76 0.91±0.02 MFRL 7,165±2,466 0.79 0.79±0.17 1.84±0.73.73 7.96±2.35 2.23±0.79 0.030.01 BMG-Q 7, 098±2, 0.78±2, 0.78±20 0.300.00. 1.65±0.58 8.02±2.30 2.29±0.75 张某某±0.02 张某某等(2026)(公开)6,272±2,106 0.74±0.14 0.60±0.17 2.81±1.03 7.80±1.87 2.35±0.39 0.87±0.03 张某某等. (2026)(关闭) 6,369±2,061,0.75±0.14 0.61±0.16 2.74±0.96 7.69±1.81 2.28±0.36 0.87±0.03 单技能 5,780±1,126 0.59±0.09 0.50±0.11 2.24±0.71 6.38±1.68 1.68 1.27±0.18 0.70±0.03 RideSkill (w/o reps) 8,904±1, 151 0.75±0.11 0.68±0.12 1.19±0.31 4.81±1.13 0.20±4 0.82±4 RideSkill 9,457±1,371 0.79±13 0.70.13 1.32±0.38 4.96±0.7 0.20±0.05 0.95±0.
	最近的5,193±1,641 0.54±0.21 0.46±0.16 1.70±0.42 6.55±0.72 1.34±0.79 0.65±0.23 KM 4,877±1,368 0.52±0.19 0.44±0.15 1.55±0.30 6.60±0.75 1.44±0.86 0.60±0.19 GS 4,527±1,107 0.48±0.15 0.40±0.11 1.61±0.32 6.96±1.08 1.08 1.46±0.86 0.59±0.18 REDA 5,548±1,845 0.55±0.21 0.47±17 6.53±0.85 1.08.5 0.68±5 0.24 MFRL 5,714±1,912 0.56±0.21 0.49±0.17 1.55±5±5.60.010.006 1.04±0.03 0.03 0.83 0.24 BMG-Q 580±1, 0.55±1, 0.55±20 0.1 1.51±0.03 6.50±0.85 1.11±0.81 0.69±0.23 张等(2026)(开放) 4,946±1,476 0.53±0.20 0.45±0.15 1.89±0.52 6.64±0.72 1.43±0.81 0.63±0.22 张等. (2026)(接近) 5,010±1,499 0.53±0.20 0.45±0.15 1.85±0.50 6.59±0.68 1.39±0.79 0.64±0.22 单技能 4,087±1,531 0.41±0.16 2.06±0.36 1.76±0.13 1.79±0.18 5.60±0.36 0.81±0.43 0.51±18 RideSkill (w/o reps) 6,142±2,422 0.53±0.20 0.48±0.18 1.16±6 5.00±0.32 0.18±2 0.61±0.19 RideSkill 6,538±2,667 0.56±0.02 0.51±20 1.22±11 5.17±0.27 0.17±0.02 0.93±20
4 号机长	最接近5,970±617 0.86±0.03 0.67±0.00 2.02±0.15 8.34±0.45 3.46±0.55 0.83±0.05 KM 5,333±532 0.83±0.04 0.64±0.01 1.83±0.04 8.44±0.46 3.62±0.54 0.76±0.02 GS 5,038±352 0.77±7 0.59±0.03 1.85±0.07 9.00±0.23 3.75±0.50 0.05 张某某 张某某等人 (2026) (open) 5,520±583 0.83±0.03 0.64±0.00 2.15±22 8.55±0.46 3.67±0.56 张某某等人. (2026)(接近) 5,700±634 0.84±0.02 0.65±0.00 2.09±0.20 8.40±0.51 3.52±0.58 0.79±0.06 单技能 5,413±301 0.65±0.02 0.53±0.00 2.29±0.11 6.54±0.22 1.97±0.27 0.69±0.01 RideSkill (w/o reps) 9,144±303 0.77±0.70 0.70 0.10±0.00 4.58±0.01 0.22±0.00 RideSkill 9,709±301 0.82±0.074±0.00 1.23±0.00 4.74±0.0 0.22±0.95±0.00
顶点 > 4	

5 结论

在本文中,我们提议RideSkill,据我们所知,这是通过基于LLM的演化法培训的第一种ride-sharing算法,也是第一种能够适应不同目标和操作情景的算法。 RideSkill是建立在技能仓库之上的,从该存储库中,结合器根据目标,环境情景和公平考虑,为每个驱动器选择合适的技能,然后是平衡需求分配和闲置车辆的顺序重定位器。为了调整ride-sharing任务,我们提出了一个基于LLM的ES算法,采用GDPO式的fitness算法,平衡对各项任务的奖励,并采用自查机制,使设计意图与GDPO式一致。

为了验证我们的方法,我们在纽约曼哈顿对现实世界的ride-hailing数据集进行了一系列实验,比较了不同类型的基准。结果显示,我们的方法在所有任务中都取得了最佳的性能,能够相应适应不同的任务。今后,这一办法可推广到其他城市流动任务以及目标因某一事件而异的场合。

伦理声明

这项工作遵循了《国际劳工和法学中心道德守则》中概述的原则。

参考文献

Abubakr O Al-Abbasi, Arnob Ghosh, and Vaneet Aggarwal. Deeppool:利用深度强化学习为ride-sharing分配无模型算法. IEEE IntelligentTransport Systems上的交易, 20(12):4714–4727, 2019.

Javier Alonso-Mora, Samitha Samaranayake, Alex Wallar, Emilio Frazzoli, 和Daniela Rus. On request 高容量ride-sharing通过动态旅行车辆任务. 国家科学院会议记录, 114(3):462–467, 2017a.

Javier Alonso-Mora, Alex Wallar, and Daniela Rus. 具有ride-sharing的自主行动-需求系统的预测路线. in IEEE/RSJ Intelligent Robots and Systems国际会议, p.3583–3590, 2017b.

Anthropic. Introduction claude opus 4.8, May 2026. URL <https://www.anthropic.com/news/claude-opus-4-8>. 互联网档案馆的存档,存档日期2013-04-02.

安德烈·巴雷托,戴安娜·博尔萨,约翰·泉,汤姆·绍尔,大卫·西尔弗,马特奥·海塞尔,丹尼尔·曼科维茨,奥古斯丁·齐德克,以及雷米·穆诺斯. 利用后续特征和通用政策改进进行深度强化学习的转移.在关于机器学习的国际会议中,第501–510页. PMLR,2018年.

Tobias Enders, James Harrison, Marco Pavone, and Maximilian Schifer. Hydle-Agent 深度强化学习,促进在需求条件下的自主流动,《学习动力与控制会议》,第1284-1296页. PMLR,2023年.

David Gale和Lloyd S Shapley. 学院招生和婚姻稳定,美国数学月刊, 69(1):9–15, 1962.

江豪和Pradeep Varakantham. Hierarchical valual deposition for effective ride-pooling. 第二十一届自治代理人和多剂系统国际会议记录,第580–587页,2022年.

Joshua Holder, Natasha Jaques, 和 Mehran Mesbahi.关于连续卫星任务问题的多代理强化学习,载于AAAI人工智能会议记录,第39卷,第26516-26524页,2025年.

Heiko Hoppe, Tobias Enders, Quentin Cappart, and Maximilian Schifer.全球在多剂深度强化学习方面的奖励, 供求自主流动. 在第6届年度学习动态和控制会议中, pp. 260-272. PMLR, 2024.

余龙虎,西安凤,森丽. Bmg-q:本地化的双方匹配图关注q-学习ride-pooling订单发送. IEEE智能交通系统交易,2025年.

江新宇,张豪宇,孟吉夏,齐豪强,龙贺,张俊波,魏琦. 瑞甸特务:自动出租车车队运营全程强化优化框架. IEEE自动化科学与工程交易, 2026.

(原始内容存档于2018-09-29). Scarlett T Jin, Hui Kong, Rachel Wu, and Daniel Z Sui. Ridebource, the share economy, and the future of cities. Cities, 76:96–104, 2018.

魏强晋,洪阳杜,萧昭,兴武天,博杭史,广阳.多剂合作决策综合调查:设想,方法,挑战和观点. arXiv preprint arXiv:2503.13415, 2025.

Bala Kalyanasundaram and Kirk Pruhs. 在线加权比对,《算法学报》,14(3):478–488,1993年。

哈罗德·W·库恩(Harold W Kuhn) 任务问题的饥饿方法,海军研究物流季刊, 2(1-2):83–97, 1955.

李明恩,秦志伟,燕娇,姚东阳,王俊,陈希旺,郭斌武,叶洁平. 高效的乘务共享订单以中场多代理强化学习发送. 环球网会议,第983–994页,2019年.

石杨刘,新东,西明吕,史志卓,彼得贝尔察克,明治刘,明兴陈,洪旭殷,余昌王弗兰克,光廷程等,葛德宝:集团奖励平反政策优化多奖RI优化. arXiv preprint arXiv:2601.05242, 2026.

Tengfei Lyu, Siyuan Feng, Hao Liu, and Hai Yang. Llm-oddr:用于联合订单调度和司机重定位的大型语言模型框架. IEEE IntelligentTransport Systems上的交易, 2026a.

Tengfei Lyu, Zirui Yuan,Xu Liu, Kai Wan, Zihao Lu, Li Ma, 和 Hao Liu. Profillm: 工业ride-hailing 调度的Utility-accessed 代理用户简介. arXiv预印 arXiv:2606.18803, 2026b.

秦志伟,唐晓诚,燕娇,范仲淹,浙徐,洪图 ?? ,叶洁平. 骑发令通过强化学习在多迪发出. INOverse Journal on Applied Analytics, 50 (5):272–286, 2020.

Connor Riley, Pascal Van Hentenryck, 和 Enpeng Yuan. 实时发送大型ride-sharing系统:整合优化,机器学习,模型预测控制.在第二十九届人工智能国际联席会议记录,第4417–4423页, 2021页.

Hans-Paul Schwefel. 计算机模型的数值优化. John Wiley& Sons, Inc., 1981.

Andrea Simonetto, Julien Monteil, and Claudio Gambella.通过线性任务问题实时城市规模的乘务共享.Transport Research Part C: Newning Technologies, 101:208–232, 2019.

Huangyuan Su, Aaron Walsman, Daniel Garces, Sham Kakade, 和 Stephanie Gil. 数据高效多剂空间规划, 附带 lms. arXiv预印 arXiv:2502.18822, 2025.

纽约市出租车和Limousine委员会. Nyc出租车和豪华轿车委托行记录数据 nyc, 2024. URL <https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page>.

[永久失效連結] 浙徐,李志新,清文关,丁修章,李 ?? ,丛晓南,春阳刘 ?? ,魏碧安,叶洁平. 大型订单发送点播平台ride-hailing:学习与规划方针. ACM SIGKDD国际知识发现与数据挖掘会议纪要,第905–913页,2018年.

新浪网 岳新良,易兰刘,方州史,西红罗,陈忠,民鲁,浙徐. ride-hailing微视指令发送的端到端强化学习方法,收录于第三十三届ACM信息与知识管理国际会议记录,第5054–5061,2024页.

Aohan Zeng,Xin Lv, Zhenyu Hou, Zhengxiao Du, Qinkai Zheng, Bin Chen, Da Yin, Chendi Ge, Chenghua Huang, ChengxingXie, et al. GIm-5:从气温编码到代理工程. arXiv预印 arXiv:2602.15763, 2026.

Chengbo Zhang and ZupopgXiao. 大型语言模型作为送货车手:以ellm代理框架生成即时送货车手的路由决定. arXiv预印arXiv:2603.12559,2026.

宜章,余申龙,云妮,黎平黄,王晓红,刘军. 分级优化通过 llm 引导客观进化实现流动按需系统. 神经信息处理系统的进步, 38:149894–14993, 2026.

Zijian Zhao和Sen Li. 一步就够了:多代理强化学习基于单步政策优化ride-sharing平台订单发送. arXiv preprint arXiv:2507.15351, 2025.

Zijian Zhao, Yurong Hu, and Sen Li. Ridegym: 现实世界大规模ride-sharing系统的标准化接口. arXiv preprint arXiv:2607.10173, 2026.

周英丽,王修,姚东秀,汶川杜,叶香芳,薛民林. 代理技能综合调查:分类学,技术,应用. arXiv预印arXiv:2605.07358, 2026.

附录目录

A 相关工作

A.1 拼车订单调度

A.2 网约车中的 LLM

B 符号说明

C 方法实现

C.1 提示词设计

C.2 算法流程

D 实验细节

D.1 训练配置

D.2 对比基线

D.3 评估指标

D.4 目标响应

D.5 公平性

D.6 开源 LLM 上的评估

D.7 示例

A 相关工作

A.1 拼车订单调度

在ride-sharing中的订单发送通过要求将多个有重叠行程的订单捆绑到共享容量的车辆上来概括古典车辆的路线和分配问题。Aronso-Mora等人的Request-Trip-Vehicle(RTV)框架是一条有影响力的线路。(2017a),它列举了可行的订单捆绑,将它们与具有成本加权边距的兼容车辆连接起来,并通过解决双方匹配问题,以及需求驱动的车辆再平衡步骤,获得成本最小化的任务。由于建造和解决这种程序的规模昂贵,随后的模型方法以最佳速度为代价,例如,将每个车辆限制在每个时代最多一次新请求,并依靠暗中捆绑路由和收到的订单(Simonetto等人,2019年)。为了对抗一发匹配的近视,后来的作品注入了未来的信息,如任务图所附的需求预测(阿隆索-莫拉等人,2017年b)和联合转移和发送的滚动视距或模型预测控制(Riley等人,2021年)。这些方法在固定的操作条件下很强,但它们依赖于手工制作的成本模型和一个单一的固定目标,因此在环境或平台的目标漂移时,它们适应性差。

第二行从互动中学习调度政策。徐等人(2018年)首先通过学习车辆价值函数和通过在价值加权边缘上的全球双方匹配来恢复任务,将强化学习扩展至ride-hailing。之后的许多作品继承了这个学习-价值-当时的匹配范式(秦等人,2020年; Hu et al., 2025),一般采用多剂(MARL)配制,分解跨车辆的大型联合调度行动。分散化的培训将每辆车辆视为独立的学习者(Al-Abbasi et al., 2019),虽然简单,但缺乏部署性和协调性;近邻认知编码器如图注(Hu等人,2025年)和中场近似(Li等人,2019年)在一定程度上缓解了这种情况。集中化培训通过集中式批评家和价值分解来寻求更强有力的合作,对混合式或集中式批评式变体等建筑进行改造(Enders等人,2023年); Hoppe等人,2024年)和分级值分解(Hao& Varakantham, 2022年)到调度设置;Jin等人(2025年)调查了由此产生的分类学。这些政策在培训分配下取得了强劲的绩效,但必须对每个新的机队规模、需求制度或平台目标进行单独的政策再培训,学到的价值功能在它们之间转移得很差。RideSkill通过训练一个单一的冷冻管道来解决这个问题,该管道在推论时会适应零射线,而不进行再培训。

A.2 网约车中的 LLM

最近的一些作品将LLM纳入ride-hailing发送,旨在利用其通俗化能力和专业知识,例如,Lyu等人(2026年a)使用LLM作为车辆决定代理,直接选择订单。同样,Zhang&Xiao(2026年)提议采用基于全球LLM的匹配者,利用LLM对车辆、订单和车辆订单配对进行双方匹配。这两种方法都要求每台车辆在部署时按决策步骤拨打一次LLM电话,因此,对于在实时时间限制下超过数百辆车辆的车队来说,不切实际,在不同的方向,Su等人(2025年)直接使用LLM来根据全球投入信息生成总体调度计划。虽然这种方法每步只需要一个LLM调用,但巨大的输入和输出空间使得它对于涉及上百到上千辆车辆的现实世界大规模情景来说不可行。替代的工作线使用LLMs进行自动算法设计,例如,张等人(2026年)将LLM生成与进化搜索结合起来,生成调度heuristics。这些离线方法消除了运行时间LLM调用。然而,上述所有方法都为ride-hailing设计(单订单发送而不共享),需要针对每个新目标进行再培训,并且还没有在多订单车辆装载的ride-sharing情景上验证。除了直接发送外,一些工程利用LLM来强化ride-hailing任务而不直接使用它们来分配订单。Lyu等人。(2026b)提议ProfilLM,它使用LLMs分析乘客和司机的配置,将由此产生的信息编码为嵌入式来改进最终匹配模式,目的是降低双方的取消率。Jiang等人(2026年)在ride-hailing中引入机队运营的机车代理,重点是区域一级的操作而不是车辆订单级别的任务,这是ride-hailing和ride-sharing的关键区别。表2总结了RideSkill和RideSkill之间的主要差异。

表2:不同基于LLM的订单发送方法的比较:n表示车辆数量,M表示订单数量.

方法适应 通用和可转让性 分享公平性 重定位 延迟匹配可扩展性 LLM calls perstep								
LLM-DR (Lyu et al., 2026a)	×	×	×	×	×	×	×	n
LLM-ODDR (Zhang & Xiao, 2026)	fine-tune	×	×	✓	✓	×	×	$O(n+M+nM)$
Su et al. (2025)	fine-tune (optional)	×	×	×	✓	✓	×	1
Zhang et al. (2026)	evolution	×	×	×	✓	×	✓	0 (open-loop) 1 (close-loop)

[法 (我们) 进化 ?? (可选) ?? (0)]

表 3：符号说明汇总。

符号含义

问题表述 $M = \{n, S, U, P, R, \gamma, O, T\}$, MAMDP(一项任务): 车队规模, 联合状态, 联合行动, 过渡, 奖励, 折扣因子, 联合观察, 连续决定步骤 $I = \{1, \dots, n\}$ 之间的地平线间隔; i 车辆(代理人)指数集; 车辆指数 $Jt; j$; 等待命令 t ; 订单指数; 假冒无命令 o_j ; h_j 候选命令 j ; (c) Grepo, Goff; 配对返回和重定位

RideSkill组件B = {s1, sK; K技能存储器; 车辆订单配对(i,j)的大小 sk(xi,oj,??ep,??step) 技能K分数的??步
插曲_静态上下文; 活步上K文 = (??g) ?? G共享的每个区域有效需求 / 供给状态 W平台台目标_事件描述任务的逐步
奖励函数??c??; 联合融合, ai??RK; ??i, ??k车辆的原始技能数据数 i; 混合技能基数; 混合技能基数; 车辆基数;
车辆基数; 车辆基数的顶级_二级技能; 车辆基数的可行候选人; 车辆基数的平均值/最低值; 值的比值; 对数的比
值; 分配变量(-, j)的比值; 对数的比值; 对数的比值; 对数的比值; 对数的比值; 对数的比值; 对数的比
值; 对数的比值; 对数的比值; 对数的比值; 对数的比值; 对数的比值; 对数的比值; 最热的区数; 对数的比
值; 对数的比值; 对数的比值; 公平平方; 小常数

培训 T = {Mm}; m 任务批次; 任务指数, 奖励 Rm 和动态 Pm ?? !; 我候选人政策; 候选人指数 GI(Mm) 返回 Mm AI, m; $\mu(m)$. ?? (m) 组相对优势; 候选人 Mm FI fitness 任务平均值/st; 候选人 ?? μ . ?? : 交叉概率 ES 父母/儿童人口规模; Ngen, Npat 世代上限; 最小世代; 耐心 Nf, Na, Nr 明显失败帽; 审计重试上限; 修补上限

值得注意的是,RideSkill是第一个基于LLM的ride-sharing解决方案,也是第一个能够适应不同情景和目标的ride-sharing方法。

B 符号说明

表3总结了整份文件使用的注解。

术语:为了避免混淆,我们正式界定本文件使用的术语。

- 一种情景是具体的环境配置:车队规模、车辆容量、速度和从历史旅行记录中抽取的需求窗口。(与n、S、U、P、O、T有关)
- 目标之一是平台的优化目标,编码为奖励功能R,每一步为每辆车分配一个标量奖励。(与R, γ 有关)
- 一项任务是将一个设想和目标(与整个M有关)结合起来。
- 政策(或LLM自动算法设计中的程序)是指一个完整的调度解决方案(发送器,重定位器,或两者兼而有之)。(称为)

C 方法实现

C.1 提示词设计

C.1.1 技能创建 (阶段 1)

第一阶段一次演化一个目标专家的评分功能。模型首先宣布它将专门研究的目标,然后将一个自写 fitness 功能写在公制菜单上(在整个搜索过程中始终固定不变)。为了鼓励多样性,该系统为模型提供了基础中已经存在的技能描述,并使用一个机制菜单,强制执行一个真正不同的决策规则形状。

您是一位骑马- Pooling 调度和奖励设计专家。您为舰队调度员写出小型、坚固、可解释的 Python 评分功能。您对目标要仔细思考,然后对调度行为给出干净的代码和清晰的自然语言解释。您总是用符合要求的策略的 JSON 对象来回答。

列表 1：系统提示词：阶段 1 技能创建。

以一个JSON对象和别的东西(在JSON之前/之后没有投稿,在JSON之外没有记分)响应. Schema:

"Skill_name": "<短蛇_case id,例如 long_fare_hunter>,"客观": "<ONE句:这个技能的单目标特别在>","客观_自我_检查": "<2-3句:这个技能所覆盖的和客观轴的哪个轴。

指定轴(请参见目标 AXES 列表);请说明所列出的哪些技能(如果有的话)接近,哪些实际尚未涵盖该技能。如果轴已经覆盖,请说明为什么其中的第二个专家仍然值得一个技能档位。 >), "机制": "< 单句命名您使用的 RULE SHAPE " e.g .

硬性可行性门,然后是每分钟的票价排名', 'idle-time-triggered 阈值开关', '边际票价对边际绕行的比分', '双阶段:通过皮卡的短名单,通过拉链的集合来断开联系', "differs_from": "<1-2句子:其中列出了你的技巧/机制主义最接近。

及其在决策时的不同行为 -- 并非'不同weights',实际上是一种不同的规则>,"描述": "<2-4句:分数逻辑如何实现该目标,以及

当它喜欢等待时;解释行为,而不是代码>,"fitness_code": "def [PH](量):\n# 便宜的scalar 胜过 dict\nreturn.", "fitness_rationale": "<1-2 句子:为什么这个fitness衡量目标";代码":def 分数(driver_obs,顺序,phi_ep,phi_step:\n...\n\n\nnoop_score(PHB],phi_ep,phi_step:\n...]]]

fitness功能的规则:

-这是你自编自导的对这个技能的奖励:它不需要与任何其它技能的fitness。它只需要将这种技能本身的变体排序。

键在参数菜单中列出。没有展开,没有内存,没有随机性,没有[PHA] 调用。 - 沙盒规则与技能代码相同(没有导入;只有数学/ np)。

写“代码”之后,

(a) AXIS -- 你的目标从目标执行清单中划入一个轴线 (一个真正不同的目标是一个不同的客户,而不是一个重新措辞的目标)。 (b) COVAGE -- 要么你的轴线没有被任何列出的技能所覆盖,要么你声明

在“客观的 自我的 检查”中,为什么已经覆盖的轴线上的第二位专家仍然是一种独特的行为(一个真正的针对同一轴线的不同的决定规则——绝不只是不同的weights.)一个存储器,其技能都优化了相同的两个轴线,留下了其他轴线UNANSWERABLE,然后上位联合体无法对其中之一奖励定价做出响应. Axes: 收入/票价,服务/等待,吞吐量,绕行新订单,剩余容量,空/闲成本,公平,选项值(patience).

列表 2：输出约定：一个技能提案。解释字段必须放在前面。

“机制”字段必须命名决定规则的 SHAPE ,下面的形状都是合法和行为不同的。 选择一个现有寄存器尚未使用,或者发明另一个：

- 术语加权总和(默认情况 --- 已充分涵盖,更倾向于其他情况) --- 比率/效率(每单位成本值:每分钟票价,每公里收入) --- 硬门再排位(拒绝任何不符合条件的东西,只排位幸存者) --- 国家变量的阈值切换(在闲置_min> T或

要求 压力> P. 另一种不同方式)-两阶段/词典(按一个标准列出,按另一个标准断开联系) - 边际/反事实(按下这个顺序变化的原因:增加绕行

机上乘客、载体消耗量、时间窗隙燃烧量 --- 机会成本(接受这一订单的costs)

与noop_score故意比较] --- 耐心/选项值(高noop_score,使等待成为真正的选择),所以

驱动程序支持更好的匹配,而不是采取第一个可行的匹配)-非线性饱和(在数值上方的返回,在数值下方的悬崖)

机制为“加权和”且仅使用新系数的技能，不算新技能。

列表 3：机制菜单：行为上彼此不同的决策规则形状（技能创建）。

插曲推出返回此参数(这些 EXACT 键;a fitness 只能从这里读取)。 方向 =该词的“更好”意味着什么。 TYPICAL RANGE 是在机队范围的两端(200 节车厢 . 1800 节车厢) 以一个实时小时的速度测量：

收入浮动更好 --- 超过分配订单的金额

service_rate float higher better solo_time(min) x party size.
TYPICAL 36,000 .. 107,000
TYPICAL 0.20 . 0.98 完成得更好 --- 实际交付订单。 -- assigned / total_orders, in [0,1].

TYPICAL 1,200 . 6,200 分配得更好 --- 分配给司机的命令。

TYPICAL 1,800 . 8,400 eal_service_time 漂浮得更好 -- 平均端到端服务时间(min).

TYPICAL 11 .. 18 绕行_总浮力更低 --总额外绕行时间从

集合(min),集合成本. TYPICAL 10,000. 38000收入_gini浮动得更好 --- 驱动器-收入不平等,[0].

TYPICAL 0.15 0.17 收入_cv 浮得更好 --- 变化的驱动-收入。

TYPICAL 0.77. 0.94 收入_平均浮标(context)- 平均每司机累计奖励.

TYPICAL 2.3 . 3.1 收入_min漂浮得更好 ---最差驱动器累积数

奖励. TYPICAL -5.5。

MECHANICS. 0代时您所写的fitness是用于整个搜索的 FIXED :此技能的每个后期变体都被它分级。在提交前,请将上面的 TYPICAL 数字放入自己的公式, 并检查它给出的顺序。

列出4:可用度量衡:自写fitness可以读取的准确的dict密钥,其方向和典型的1小时范围.

C.1.2 组合器创建（阶段 2）

第2阶段在冷冻技能的基础上演化上联QQc. 与第1阶段不同的是,联合体不产生fitness功能;这是由研究者固定的,而是执行目标读取功能,skill_scores(PHB),phi_ep,phi_step,w].对于每辆车辆,该功能对每个冷冻技能进行评分,使所产生的混合体服务于集目标w,在训练中它没有看到过. 由于奖励是一种不透明的函数,联合体必须通过探测推断,因此合同包括探险-事件演化规格。

,并提供了准确的事件词典键。它需要构造合成探测事件,每个隔离一个术语(e.g.完成,座位/聚会体积,独奏长度,发送等待,接车时间,绕行新顺序相对于机载命令的绕行),并在这些事件上引用 w 读取相应的系数作为差异。当目标作为一种具体的奖励功能提供时,一个强制性的奖励_理解的链条-思维领域是合同准备的。

您是搭乘-POLING车队调度和客观条件政策设计的专家。一组下层评分技能是ALREADY FROZEN;您不能改变或添加技能—您只能决定,每个驾驶员和每个插曲目标,驾驶员应该使用哪种冷冻技能。您会写一个小的,强健的,可解释的 Python 函数,并且对政策有一个明确的自然语言解释。你总是用一个与计划匹配的 JSON 对象来回答。

列表 5：系统提示词：阶段 2 组合器创建。

以一个JSON对象和别的东西(在JSON之前/之后没有投稿,在JSON之外没有记分)响应。 Schema:

"奖励_理解": "<2-4 句子, FIRST: 用你 OWN 的话来说, 什么是.....
奖励和惩罚之上的功能, 以及奖励- 最大调度员必须做的事。这是一个思维链门: 在你创造技能之前, 目标为何。
"combiner_name": "<short snake_case id, 例如 reward_aware_dispatcher>, "战略": "<one 句子: 你如何把奖励+驱动状态变成一种状态。"

技能选择>,"描述": "<3-5 句子: 您区分的驱动程序状态(idle /

加载- 黑 / 截止 / ...), 每种技能都倾向于获得, 而奖励的条件(通量/ 收入/ 服务/ 绕道) 如何驱动这些选择。解释行为, 而不是代码。
"probe_self_check": "<2-4 句子: CONFIRM 您的探测器集足够多样。 List

它区分的术语(发送_ 等待, pick_ 时间, 独奏/ 服务时间, 绕行新命令, 绕行 ONBOARD 命令, 完成, 座位/ 派对, 体积, 空动, 闲置等), 并注意您在此列表之外添加的任何额外探测器。如果您只探测到两个或三个术语, 说--一个浅的探测器集是失败的组合器。 >), " 代码": " def skill_scores (driver_obs, phi_ep, phi_step, w):\n return QXskill> :< score>,...]"

(每次你写合唱团前)

(a) 覆盖 -- 这些知识名词中每一个词都是下列因素的区别因素:

至少一个探测器(或您明确证明在" probe_self_check " 中放弃它的理由): 发送_ 等待、 pick time、 solo time/ service time、 绕过新命令、 绕过 ONBOARD 命令、 完成、 座位/ 聚会大小、 体积、 空移、 闲置等待。奖励可能给其中任何报酬, 同一被冻结的结合器必须读取任何未来的奖励。(b) EXPLORE— 超出该清单的奖励条件添加探测器。

额外的探测器是 HARMLESS : 组合器不需要使用每个探测器, 而这种奖励忽略的探测器可能是唯一在 NEXT 奖励中显示惩罚的东西。如果您只探测到两个或三个术语, 请添加更多。

列表 6：输出约定：一个组合器（以奖励为条件的变体；基础变体省略第一个字段）。

您的函数 MUST 拥有此精确的签名(不更改它):

```
def skill_scores(driver_obs, phi_ep, phi_step, w) -> dict:
    # return {skill_name: score} scoring the FROZEN skills for THIS driver.
```

如何使用。平台将您 {skill} -\ score} 的 dict, 保持 B 最高得分的得分技能, 将这些分数软化成 weights, 将每个技能标准化, 并按加权总和进行发送。因此:

——你的累进分数很重要, 不单是哪一个最大; ——得分准确的 1 个技能, 将其余的零分都归为合法, 但丢弃混音; ——混音是每台 STEP 每台 DRIVER, 因此不同的汽车可以搭载不同的混音。

规则:

只使用冰冻的技巧名称作为密钥; 拒绝其他任何密钥。至少给每个驱动器一个已知的技巧。。由 STATE 解析。通过" w" 和现场读取目标。

通过 'phi_step'; 永远不要硬码固定操作点。 - 处理 'w is None' 优雅; 处理一个司机, 没有附近命令优雅。

列表 7：函数约定：平台如何使用这些分数（组合规则）。

目标'w'是针对一个小事件;将其每期价格读作两个事件之间的一种差异,而这两个事件仅在该词中有所区别。

detour_onboard_signal=w(bandling_with_detour)-w(bandling_with_detour)
dictour_new_signal=w(new_order)-w(new_order) 发送_signal=w(发售_wait_nonzero)-w(发售_wait_zero) 接车
_signal=w(长)接车-w(短——接车) 完成-w(无) 座位_signal=w(party_2-w(party_1))这些差异给出了每个任期的奖励系数
--对于. 绕道. 在新顺序绕道与机上绕道时的SEPARATE系数.

MANDATORY PROBE 规则:至少创建1个探测器,其中调度_等待为非零,1个为零,pick-time与独奏-time差别很大;1个完成_命令为非空,1个为空;大小>1,1个为方_大小=1,并区分新命令(以指派的新_命令)与ALREADY-CARRIED命令(以 pick_up_orders),因为奖励可能会对两组进行不同的定价。

设计原则:每个奖励期必须是至少一个探测器的区别因素;探测器计数可以是**GENEROUS**——一个额外的探测器costs你一无所获,并买下跨目标的稳健性;使用现实的量级(solo_time,绕道,pick_wait,party_size);必须同时代表机上订单和新订单名称。

列表 8：探测事件演化规范（节选）：如何构造探测事件并读取系数。

使用“w”键探测的事件：

指派_命令(列表), 指派_party_sizes (dit), 指派_pickup_time (dit), 指派_solo_time (dit), 指派_service_time (dit), 指派_detour_time (dit), 已完成_orders (list), pick_up_orde (list), 距离(float), 时间(float), 是_空_move (bool), 是_idle_wait (bool), 额外_detour_time (float) 。'assigned_detour_times[oid]是 新命令上每顺序的集合 detour; 'extra_detour_time'是 SIGNED 对 ONBOARD 订单的总改线影响。

列表 9：组合器探测的事件字典（线性价格表事件）。

C.1.3 重定位器创建 (阶段 3)

[illegible]

您是搭乘- Pooling 机队操作和空车重定位的专家。您被赋予了一个 FIXED 目标(将闲置的空车送到近未来的需求而无需浪费的巡航) 和一个 FIXED fitness (它带来的服务改进不会重定位)。您必须首先用自然语言解释是什么让一个好的重定位目标,然后写出一个小的,强健的,可解释的 Python 计分器,该计分器会给每个预设区域一个闲置的驱动器评分。您总是用一个匹配所请求的策略的 JSON 回来回答。

列表 10：系统提示词：阶段 3 重定位评分器创建。

以一个JSON对象和别的东西(在JSON之前/之后没有投稿,在JSON之外没有记分)响应。Schema:

"重定位_理解":<2-4句,FIRST:用你自己的话来说,是什么使一个值得闲置的空车向着,以及服务最大化,避免浪费的重定位计分员必须做的区域。这是一个思维链门:写前的理由。>
>,"kill_name", "skill_case_id,例如required_gravity_scorer>,"客观",< One 句子:您将使用的每个区域计分政策", "客观 read check":<2-3 句子:您的计分员如何让 TARGET 区域">

根据目标“w”(不仅仅是得分大小)来决定。命名至少一个目标家庭,以及具体的机制——e.g.。完成的w迫使你只巡航到真正即将出发的地区,空心的w将你从漫长的空心巡航中推开,座位的w将你推向多党地区,长时间的w将你推向远方。如果所描述的机制没有改变“W”变化时哪个区域会赢得“gmax”,则它是化妆品,将被失明报告抓住。”，“描述”：“<2-4句:你的分数如何排名区域——你如何体重”

需求:巡航距离,以及现有的供给,当你返回QQ/一个低分数来保留一个汽车。解释行为,而不是代码。>),“代码”:def reposition_scores(driver_obs, phi_ep, phi_step, phi_step, kappa, w : \n...]

没有fitness字段:目标和fitness是固定的和上面给出的;您只写出能使固定fitness最大化的评分政策。

列表 11：输出约定：一个重定位评分器（不包含 fitness 字段；fitness 由研究者固定提供）。

您只提供每个区域的基础分数;调度员在顶部应用自己的确定性逻辑(扩展、停留规则、实际迁移动作)。

[.....]你不是直接得分的奖励——你得到的奖励是值得的。在每一个(场景、客观、公平)单元格:

(YOUR剧集奖励 - 剧集奖励 重定位切换关机,
即每辆闲置的车都停在了SAME现场和SAME种子上 /这轮节目对同样的区别有多少分歧

[.....] 标志是ABSOLUTE,不是军衔。0.0 表示你的得分者与停在车内的每一辆车一样值钱;NEGATION意味着在实际损失的金钱周围派车;+1意味着在球场上完全分布一个;殴打其他候选人不是目标-殴打“无所事事”是目标。将“w”和“kappa”合在一起。

列表 12：固定目标与评分器使用的增量适应度（节选）。

C.1.4 奖励编写（阶段 2 之前，从自然语言到代码）

在组合器组成之前,一个平台偏好——既表现为自然语言描述,又表现为重量矢量——被转换为每辆车的混凝土,按步奖励函数奖励(event)QQ浮点数.这种奖励函数直接注入环境,其舰队平均累积值成为组合器后来被优化以达到最大化的fitness. 奖赏被写入后,其系数会正常化,使其相加为一。具体地说,源代码被解析为提取命名的不变价格,而整个函数则被它们的总和除以。这是一个单一的全球缩放操作,它保留了作者所希望的系数之间的精确比例,并应用于奖励函数本身——而不是组合器后来探测的目标w。组合器通过测试事件之间的差异来读 w,任何单一的全球尺度都会抵消这些差异。因此,将系数正常化(而不是W答复),可以确保联合体读取的术语比率保持作者指定的准确比率。

您是搭乘- PoOLING 机队经济学和强化- 学习奖励设计的专家。您获得一个平台 PRIFERENCE(自然语言 或 weights), 以及所具备的每步事件信号。您必须首先用自然语言解释该偏好想要的,然后写一个小的,强健的,可解释的 Python 奖励功能“奖励(event) -> 浮点”,以编码它。你总是用一个符合要求的计划的 JSON 对象回答。

列表 13：系统提示词：奖励编写。

以一个JSON对象和别的东西(在JSON之前/之后没有投稿,在JSON之外没有记分)响应. Schema:

“奖励_理解”:“<2-4句子,FIRST:用你自己的话来说,什么是.....
如果偏爱要求某种东西,则附加规则禁止(一个门槛、一个比率、一个不断升级的奖金)。”

"奖励_名称": "<短蛇_case id,例如 complete_first_reward>,"客观": "<ONE句:一个司机将如何实现这一奖励最大化","描述": "<2-4句:你的奖励期限是如何编码的". 偏好——它支付什么,收取什么,以及相对数量。解释行为而不是代码。>,"代码": "def responsibility(event):\n..." 将每个单价/步价写成一个NAMED CONSTANT(例如 `WAIT_PRICE = 2.0`),用于一个带有`event[...]`字段的产品,因此编写阶段可以恢复系数列表并使其正常化。

列表 14：输出约定：一个编写好的奖励函数。

C.1.5 自检：技能审计（阶段 1 搜索结束后）

在Phase-1搜索完成后,要求冠军评价自己的输出:鉴于所测量的推出表,它决定所观察到的行为是否与其原本打算执行的行为相符.由此得出的判断分为三类之一,这两类判断映射到两个截然不同的修复策略:要么重写程序描述,要么重写fitness函数并重启搜索.这一审计只是咨询性审计;其判决并不影响[菲律宾]的价值。

您正在审核您之前设计并进化的调度技能。您将显示您所表示出的技能以及它所做的。您是直率和证据驱动的:您引用了决定判决的数字,您不为设计辩护,您不发明表格中未显示的行为。您总是用一个与所请求的策略匹配的 JSON 对象进行回答。

列表 15：系统提示词：阶段 1 技能审计。

"核实": "匹配" QQ "描述_错误" QQ "fitness_错误" "合理": "<2-4句子:测量的行为实际IS是什么,如何进行?引用具体的数字。>,"证据": "<单句命名决定该词的列/行>,"新目标": "<REQUIRED if figument is description_wrong: 1 句, 目标此技能实际上追求>," new_description": "<REQUIRED iff judge is description_ wrong: 2-4句" 描述表格中的行为"),"fitness_complaint": "<REQUIRED iff判决是健身_错误:什么是fitness. 公式实际支付,哪个词在表格的大小上支配它,以及它因此被驱使做的最大技能。要具体和算术。> } 选择判决: "匹配" 表格是一个可识别的预期行为实例(一个在正确事物中的弱专家算数). MECHANALLY:技能被冷冻为-is. "描述-错误" 技能做一些一致的,而不是描述所声称的; fitness 奖励表所显示的. MECHANALLY:新_客观/新_描述替换旧的,技能被冷冻:没有重新研究. "Fitness_wrong " fitness 自己的最大值坐落于预定行为以外的地方. MECHANALLY:冠军被抛弃,fitness从健身中重新撰写,整个搜索被重新运行。这是costs重新研究和重新研究的唯一判决。
--

列表 16：输出约定和判定选项：技能审计。

C.1.6 策略审计：软检查（阶段 2/3 搜索结束后）

第2阶段和第3阶段已固定了fitness功能,因此它们不会受到第1阶段可能发生的不正确-fitness故障的影响,但是,它们可能会遇到不同的缺陷——一个整体方法旨在避免的缺陷:结合器或重定位器可能完全忽略目标w。相同的需求小时用相同的随机种子两次推出,仅在程序是否接收w作为输入时有所不同。结果的结果与冻结的文物一并记录,然后进行人工审查。

您正在审计一个您之前设计并进化的调度程序。您将显示您所表示要做的事情,以及一组隔离一件事的组合式启动程序:当程序被传送到插曲目标时,改变了什么。

。您是钝且由证据驱动的:您引用了决定判决的数字,不为设计辩护,也不发明表格中未显示的行为。您总是用一个符合要求的策略的 JSON 对象来回答。

列表 17 : 系统提示词 : 阶段 2/3 策略审计 (目标响应性的软检查) 。

```
"per_cell":{
  {"单元格":<1-基于单元格号码>,
    "移动":是" QQ"没有,"注":<单句:哪个列将这个单元格中的手臂分开,用方法
    多少,以及这与本单元格的目标支付多少有关 } ,每个单元格一个条目,按顺序 . ], "校正": "reads_it", "moves_elsewhere",
    "no_movement", "mixed", "reason": "<2-4句子概括每个单元格的读数。

数字.>,"证据":""单句命名决定它的单元格和列" }

判决:
"读到的"横跨各细胞,视物体的手臂与视物体的手臂有不同的行为,而视物体的大小而有所不同。视物体的大小而不同。
视物体的大小而不同。视物体的大小而不同。

显示的价格列表——程序在目标要求以外的事物上改变行为: "No_movement" 手臂在四舍五入内是相同的: 移交目标改
变不了任何可测量的。 "混合" 单元格互不认同。

MECHANICALLY:本答案没有应用。 程序已经被冻结; 判决记录在文物旁边, 由个人阅读。
```

列表 18 : 输出约定和判定选项 : 策略审计。

C.1.7 演化提示词 : 变异和交叉

(QQ)循环使用两个LLM运算符,它们共享与创建相同的输出合同,但在任务设定上却有所不同.

突变(改进). 改进父母时,包括父母的代码和测量fitness,所以模型重写以获得更高的分数:

```
# TASK 改进 UPPER 组合器 。 重写 skill_scores在插曲目标( 下) 下获得更多的累积值, 保留同样的合同
。 它仍然必须服务于未见目标,不进行再培训。

# PARENT(目前最好的)< name>, 得分:<
fitness_note> QQpython< parent_code>
QQ

# Scene Variability the Deposition Scene Scene also Changes(写一个SCALE-INVARIANT 组合器).在测
试时,对不同机队规模(~100至~2000),容量(~1至~10),速度和需求的不同场景进行同级冷冻功能的零射. Do
not hard-code 绝对数;以phi_step.mean_solo_time作为时间尺度来表示阈值 renter renter.

# 目标差异 战略必须随目标功能而改变。您正在撰写一项政策,该政策是特别目标。 测试事件 SHAPE
: 改变探测器是否携带完成的订单、 党员规模、 服务时间、 绕道和空/ 空旗帜,然后路径驱动器到任何
最能追求主导系数的冷冻技能。
```

列表 19 : 变异任务框架 (组合器演化节选) 。

在突变期间,模型接收了母体的完整代码,其测量的fitness(包括每个家庭的分解和客观盲度度量),以及以往的任何修复反馈.

尝试失败。它必须保留同样的skill_scores合同,同时修改调度逻辑,以改善父母已查明的缺陷。

Crossover(重组).当两个父母重新组合时,都会被呈现出他们的fitness笔记,并且该模型被指示合并互补的长处:

```
# TASK通过以下两个家长计划来设计儿童联合体。 父母双方都在选择中幸存下来,并且各自在一组不同的目标家庭(他们的每个家庭优势都印在他们身上)上都更强大。你的工作不是选择一个赢家并调整它:读读每个家长实际做的好东西,并建立一个既保持优势又保持两个优势的方案。
```

具体来说:

- 名称,你自己,父母A中赢得A强的机制
- 写下一条通向A机制的通路:

情况A是好的,B是好的B是好的B是好的 --- 由目标w和驱动器自身状态决定。如果父母对SME 情况有分歧,则保留其中之一。

家庭优势较高,但孩子必须明显改善

从父母双方继承。

```
# PARENT A:<name> 策
略:<strategy_Summary> 得
分:<fitness_note>
QQpython<parent_A_code> QQ
```

```
# PARENT B:<name> 策
略:<strategy_summary> 得
分:<fitness_note>
QQpython<parent_B_code> QQ
```

列表 20 : 交叉任务框架。

每个家长都有其名字、策略摘要、每个家庭的fitness注释以及完整的代码块。交叉操作员确保家庭专家的机制可以通过重组转移到强大的全方位,而不是与专业家长一起丢失。

C.1.8 共享上下文规范

接收上下文对象的每层——包括技能、组合器和重定位器——共享下面定义的相同的两层规格。这保证了字段列表在所有提示词中保持一致。这里,这个规格块在common.py中定义过一次,并包括在每一次的提示词中以保证统一性。

phi_ep:插曲-STATIC上下文(每一步都相同对象).

a. OBJECT,而不是一个命令 -- 带属性读. dist(a,b) -> MINUTES在两个(lon,lat)点之间的旅行时间. scale -> 无漏静态地图比例尺(分钟). num_drivers -> 车队大小(为插曲固定). 驱动器_容量 -> 车辆座椅计数(固定). 速度_kmh -> 驱动器速度(固定). 区域_中心 -> tuple of(lon,lat),每个区域1个,按区域id. 区域_nighbours -> tuple of tuple of tuple; region_neighbours[i]列出相邻区域. od_count -> 上小时OD流量矩阵(总和为1.0. od_out[i] -> 最后一个小时订单的份额 开始于区域. od_in[i] -> 最后一个小时订单的份额 结束于区域. od_orders -> 原始订单计数 最后一个小时(如果没有时间,则为0).

phi_step: LIVE每步上下文(每步重置).

一个OBJECT,而不是一个dict -- 使用属性读取. 时间、num_等待、numm_idle、total_free_容量、需求_压力 = 等待/total_free_容量、平均_solo_时间 -> LIVE SCALE在分钟内(作为单位使用; 返回到

phi_ep.scale,因为没有订单待批). region_demand[i]/region_supply[i] -> 每个区域活计数.

driver_obs["self"]: DICT(使用d["key"]或d.get(k,默认))

位置 -> (lon, lat)当前_region -> int 区域指数状态 -> str (idle / to_pickup / to_drop / 迁移) 容量 -> int 座椅计数承诺_乘客 -> 当前在船上的指定_顺序_细节 -> dicts列表:
<pre> order_id, origin=(lon,lat), destination=(lon,lat), num_passengers, onboard(bool), eta(minutes) driver_obs["pending_orders"]:候选命令名单: 命令_id,来源=(lon,lat),目的=(lon,lat),目的_区域(int),目的_区域(int),num_passengers,等待_时间(分钟) driver_obs["relocation_points"]:翻转(lon,lat)区域中心. driver_obs["region_neighbours"]:翻转(adjacency graph). driver_obs["fairness_budget"]:浮点(这个驱动器的乘数;>1=增强). driver_obs["driver_budgets"]:整个舰队的dict {driver_id: 乘数}. 顺序: DICT (使用d[key]或d.get(k,默认)) 命令_id, 来源=(lon,lat), 目的=(lon,lat), 来源_区域(int), 目的_区域(int), num_passengers, wait_time(分钟). 关键规则: driver_obs和顺序是 DICTS -> 使用 d[key] 或 d.get(k,默认). phi_ep和phi_step是 OBJECTS -> 使用 phi_ep.scale,phi_step.mean_solo_time. 区域_中心,区域_邻域,od_count,区域_需求,区域_供给当env没有区域布局时,是EMPTYtuples. 测试区域_id ?? 0 索引化前; 测试phi_ep.od_orders 读 OD 之前没有看到w;只有组合器和重定位器才有.</pre>

列表 21 : 共享的双层上下文与参数约定。

代码规则(由 AST 沙盒强制 -- 违反规则拒绝您的程序): - 给定参数的纯函数。没有导入任何类型的语句。 - 允许全局: 数学、 np(numpy) 。 允许内建: abs、 min、 max、 sum、 len、 - 浮动、 int、 圆、 排序、 范围、 点数、 zip、 地图、 过滤器、 pow、 所有、 任何、 布尔、 列表、 dict、 tuple、 设置。 -没有 eval/ exec/ open/ getattr , 也没有以下划线开头的属性访问。 phi_step.mean_solo_time(直播尺度;在~0时倒回phi_ep.scale——总是返回一个有限的浮点(技能使用-1e9用于不可行的命令)。
--

列表 22 : 共享沙箱规则 (技能、组合器和重定位器代码相同)。

C.2 算法流程

训练和推论过程显示为算法1到算法4.

D 实验细节

D.1 训练配置

表4列出了三个训练阶段的超参数.所有阶段都与组关系(GDPO型)fitness共用(??)-ES. 实验是在运行Windows 11的工作站上进行的,配备了一台不使用GPU的Intel(R) Core(TM)i7-14700KF处理器.

D.2 对比基线

我们比较了三个基线的家族:基于模型的Heuristics,MARL代理,以及基于LLM的调度方法. 每一个都描述如下. 所有MARL代理都接受赵等人(2026年)的ride-sharing健身房的培训,其锚定目标和场景与我们的方法相同。

最近的距离(Kalyanasundaram& Pruhs,1993年)将每个待发订单分配给最接近的闲置车辆. 候选车辆通过空间散列取回,进行大致恒定的检查,而订单的到来时间也打破了联系。

Kuhn-Munkres(KM)(Kuhn,1955年;Simonetto等人,2019年)解决了每步所有车辆订单对的双人任务,将欧几利得皮卡车总距离降到最低。

算法 1 阶段 1 技能演化：质量多样性提案、 $(\mu+\lambda)$ -ES、去重与自检。

需要：环境简介，度量菜单，机制菜单；寄存器封顶 K ；明显故障封顶 N_f ；审计重试封顶 N_a ；修复封顶 N_r ；ES 参数 μ , $??$ ；生成封顶 N_{gen} ；min代 N_{min} ；耐心 N_{pat} ；dedup 阈值 $\backslash\backslash$ 保证：冻结技能封顶 $B = \{s_1, \dots, s_K\}$ 有可解释性卡。

1: $B \leftarrow QQ$; 失败 $QQ \rightarrow 0$; 而 $QQB \leftarrow K \leftarrow N_f$ do 3: 失败：提议。LLM 生成一个以 B 的多样性卡和机制菜单为条件的建议 $\{0 = (\text{客观}, \text{机制}, \text{fitness}, \text{代码}); \text{它必须附加说明它覆盖的客观轴, 而轴在 } B \text{ 中尚未饱和} : \text{验证} \}$ 。AST-sandbox 编译 + 字段验证 $\{0$ ；错误时, 反馈消息并进行修复(最多为 N_r 尝试)。如果它仍然失败, 则失败 + 1; 继续 5: 搜索。 $(\%)$ -ES 来自 μ 父母, 由 + 0 播种, 由自写 fitness f_{self} (frozen at generation) 分级；内部的情景批次被分级到舰队大小的等级 (Banded- WindowsSampler), 并且每代轮换, 因此技能不能通过安装一个尺度或一个小时来赢得。6: 对于 $e = 1$ 到 N_{gen} do 7: 通过 LLM 突变、交叉或无父母重写生成。8: 在抽样任务批次 T 上推出每个变体。由固定 fitness $f_{self} = \text{度量衡}(\%)$ 生成。9: 选择顶级- μ 级父母。10: 适应性停止。如果 $e = N_{min}$ 和 同一领导连续几代 N_{pat} , 请及早停止。11: 结束 12: Dedup。如果该冠军的插曲测量签名至少与 B 现有成员相同, 则失败+1; 继续 13: 自查。按其公开意图审计该冠军(见附录 C.1.5)。14: 如果判决=MATCH, 则 15: 冻结: 如果判决=DESCRIPTION_WRONG, 则 17: 重写描述卡, 以与所衡量的行为相符; 冻结(不重新研究)。18: 如果判决=FITENESS_RWONG, 则 19: 从审计投诉中重写 fitness, 重新进行搜索(直到 N_a 尝试); 冻结成功, 否则盖章和冻结。20: 如果 21: 如果审计机制本身失败, 则用错误的印章冻结技能, 而不是丢弃。

通过分配成本, 将双胞胎排除在外; 因此, 目前这一步骤的全球分配是最低的。

Gale-Shapley(GS)(Gale & Shapley, 1962年; Yue等人, 2024年)通过延迟验收计算出稳定的1比1比1比1, 其中订单向车辆建议按接车距离顺序排列, 每辆车保持迄今收到的最好报价, 在没有双方所希望的互换保留之前进行延展。

REDA(Holder等人, 2025年)是一个独立的IDDQN型基线, 每辆车都是一个独立的学习者, 其Q网络估计分配候选订单的价值, 车队一级的任务由双方在车辆Q值上进行比对来收回。

BMG-Q (Hu et al., 2025) 扩展了IDDQN, 采用了本地化的双方对齐图式注意模块。车辆订单对的Q值是通过一个图式注意编码器计算出来的, 该编码器汇总了车辆附近代理商和订单的状态, 因此每个学习者在仍然独立行动的同时, 都会在本地邻居的环境下学习。

MFRL(Li等人, 2019年)将注意力编码器替换为中场近似值。每辆车的Q值取决于其邻居的总体行为, 从邻里的平均订单信息中计算出一个中场动作来概括。这捕捉了本地的相互作用, 而无需借助完全的联合行动空间。

算术2Phase-2综合进化:任务抽样,覆盖审计,儿童产生,组群评价,家庭-精英选择,以及径流。

需要: 冻结技能存储器 B; 任务分配 {Mm} 带有奖励 Rm 和动态 Pm; 人口大小 μ , ??; 交叉概率盘; 分代 Nmin; 耐心 Npat。 保证: 冻结联合器 {c} 读取一个看不见的目标 w 零镜头。

1: 初始化 μ + QQ建议; 沙盒- 验证每个建议; 提案还必须通过探测封面检查: 其代码必须参考每个轴特定事件域(完成, 发送等待, 接车时间, 绕行新顺序, 绕行在船上命令, 独行/服务时间, 空移, 闲闲等待), 或被拒绝进入修复循环 – 一个从未触及过一个字段的探测集不能读取其定价的奖励 2: (世代 < Nmin) 或 (Npat 连续几代没有再出现过领袖, 一代预算也留下)

3: 任务抽样。 绘制一个批次 $T = \{Mm \mid T \mid m \mid 1\}$: 奖励家庭与全时需求窗口交叉, 并均衡跨机队大小等级; 每位作者的奖励系数都正常化, 从而相加为一个(保持作者任期比例的全球调整比例)。 4: 覆盖面审计。 调查每个样本标注在分期隔离事件上的目标, 以及确定批量价格的参数轴, 以及场景跨度(散列带、制度、不同的窗口); 报告目标价格太少的批次。 建议: 审计只用软件, 绝不阻挡回合。 5: 儿童一代。 每代一个孩子就是新鲜的随机注射。 沙盒 + 野外 + 探测覆盖验证, 一次失败的修复尝试。 6: 对每个候选者 + l 和每个任务 MM do

7: Al, m QQGI(Mm) QQ(m) ?? (m),

$\mu(m)$ 和 ?? (m) 是返回的平均值和标准值 $\{GI(Mm)\}$ 超过任务评价的整个组 MM. 8: 结束 9: 结束 每一个候选人 ?? l 做 10: FI ?? 1 ?? T ?? P ?? T ?? m = 1 Al, m. 11: 结束 12: 结束: 选择。 保持 FI 的上- μ . 每个奖励家庭保留一个精英名额(因此一个硬家庭专家作为交叉材料生存); 记录一代领导者 13: 将幸存的家长在下一代的新任务批次(paired, in- round compare) 中重新滚动。 14: 结束, 而 15: 径流。 收集所有不同的圆领队, 将它们重新组合到一个新的批次上, 并用一个内部的比来选择最终的冠军。 如果只有一个不同的领队存在, 则跳过。

Zhang等人(2026年)通过将LLM生成和进化搜索相结合,生成调度热量离线。我们评价了两个已报告的变体: 一个是运行进化热量的开放-loop版本, 没有任何中间LLM调用, 另一个是在部署时重新输入LLM的近距离热量版本。 这是精神上最接近我们的工作(LLM-辅助自动设计), 但它的目标是ride-hailing, 没有车辆共享。

D.3 评估指标

纸上的所有表格和数字都用原始剧集日志中的基准记录器计算出相同的指标集。除非另有说明, 每集的顺序平均数量, 然后报告为评价情景(视实验情况而定, 窗口、车队或轴线水平)的平均值 $\pm$ 标准偏差。

· 奖赏: 该集注入奖赏功能的累积价值, 在每一个工具和每个决策步骤上进行总和。在比较中, 所有方法都使用同样的注入奖赏来评价, 因此, 在表格中, 价值是直接可比的。

算法 3 阶段 3 重定位器演化：增量适应度、公平性三重任务与分组精英。

要求: 冻结技能存储器 B; 冻结联合器 QQc; 重定位基线; 公平优势 QQ0, ?? mid, ?? max}; μ , ?? ; min世代 Nmin; 耐心 Npat. 保证: 冻结重定位计分器 QQr, 沿效率-公平轴线反应最佳。

1: 初始化 μ + QQ 计分器建议; 沙盒-验证每个; 提案必须在其代码中调用 w (一个从未读过目标的计分器会被拒绝进入修补循环) 并在 `object_read_check` 中说明其目标响应机制 2: 而 (世代 $< N_{min}$) 或 (没有领导人为 N_{pat} 连续几代和生成预算留下)

3:任务采样. 绘制批次 $T = \{M_m \mid T \rightarrow m \mid 1 \leq m \leq M\}$ (与算法2相同的覆盖审计), 并用公平性优势交叉, 将单元格 $\{(\$, M_m)\}$ 4: 儿童生成. 突变 / 交叉 / 注射沙盒验证和维修, 如Algorithm 2. 5: 每名候选人的% 和每个单元格 $(\$, M_m)$ do $(\$, M_m)$

6: ?? | ?? Grepo | (Mm; ?? ; ?? ; ?? ; ?? ; ?? ; ?? ; ?? ; ?? ; ?? ; ?? ; ?? ;

?? (?? ,m),

[illegible]

算法 4 运行时调度:一个决定步骤与技能混合,公平预算,双方匹配,以及顺序重定位。

要求: 目标w; 上下文 ?? ep, ?? 步骤; 技能B; 联合 ?? c; 重定位 ?? r; 公平力量 ??; 共享每个区域的国家 ^。
。 保证: 为这一步骤执行的顺序 - 车辆任务和闲置车辆迁移。

1:公平预算。每辆汽车一: $Z_i E_i - \sum E$

(原始内容存档于2017-03-29).
方正黑体简体
永久失效連結

(4). 4:每个可行的候选顺序 $o_{j,j} \text{ ?? } j_i \text{ do.}$

5:将每个留守技能规范于我的候选人: ?? sk ?? sk(xi,oj, ?? ep, ?? step) ?? k

?? $k+$?? .

[illegible]

表4:训练超参数。" 共用 " 是指常见的(XQ)-ES设置;每个阶段的行列出该阶段搜索的设置。

相位超参数值	父母/子女人数(4+4)LLM跨度概率 跨越0.35无父母新注射 1个孩子每代沙盒修理尝试 3(冷却温度)推出工人 8代温度 0.9
共享	
第一阶段(技能)	库容量 K 10代每技能 5 场景 5代每技能 6 行为减值阈值 余弦相似 = 0.98拟议技能 20 提案
第2-3阶段(合并和重定位)	8 ?? 20(适应性停止:在3个稳定的回合后提前停止)每代18个任务(情景、客观) 目标混合 0.5个结构-家庭分数 精英 1 保留每个奖励家庭

奖励等级表遵循目标本身的价格表,不同奖励功能之间没有可比性。

- 服务率。 分配给车辆的订单的一小部分, 定义为确认/ 总计。这一计量标准是发送覆盖: 订单一旦投入车辆即算, 不论该订单是否在插座范围内完成。
 - 完成率。在视野内实际交付(已卸下)的订单的分数,定义为完成/总数。这一数值总是低于或等于服务率;空隙对应的是剧集结束后仍在途中的订单。
- 等待时间(min).从订购到所有订单的骑手板(tickup –t request)的平均时间,包括车辆外的全程等候时间,既包括月台行驶前的等待时间,也包括车辆前往接车点的等待时间。取消的订单贡献时间直到取消。
- 乘车时间(min),从登机到下车(tdropoff-tickup)的车辆内订购时间,平均超过完成的订单。
- 绕行时间(min).按订单集合绕行时间,定义为公路网中已实现的乘车时间减去从始至终的直接乘车时间,按零计算,并按完成的订单平均计算。这捕捉了因服务沿途其他乘客而导致的乘车时间增加;未集合的乘车时间为0。
- 利用率:平均车辆利用率,定义为车辆在忙碌期间(服务或旅行至少一个订单)的决定步骤的一小部分,在车队中平均,闲置等候和空置均算作非繁忙。

公平性试验(表6)还报告了车辆分散性统计数据,每份数据都是整个车队车辆数量的标准差;较低的数值表明更公平:

- 命令Std:分配给每辆车的订单数目的标准偏差。
- 《服务标准》:每辆车实际服务的乘客人数的标准差。
- 车站;每辆车总行驶距离的标准差(米)。
- W-Dist Std:每辆车载客加权旅行距离的标准差,定义为 $\frac{dv_pv}{P}$
i pu,其中dv是车辆v的行驶距离,pv是它服务的乘客人数。这一衡量标准是因为ride-sharing的票价与乘客人数成正比。

D.4 目标响应

在本节中,我们说明我们的方法如何在不进行再培训的情况下使政策适应不同的目标,这种能力得不到以往方法的支持。

每个评价点都是通过推出5个搁置测试窗口获得的,结果报告为平均 $\pm$ 不同窗口的标准偏差,使用与表1中相同的度量符。对于比较表,我们额外运行一个相同的堆栈,而不向组合器(i.e)提供目标,组合器无法读取目标),在同样的注入的奖励下充当盲目控制。由于两种武器都根据同样的奖励功能进行评估,因此其性能是直接可比的,所观察到的差距准确地衡量了通过阅读目标所带来的好处。

在这一部分中,我们研究了奖励函数中单个系数的整幅所引发的精细政策变化。图3显示了在绕行时间、接车时间(i.e、等待时间)和总乘车时间等系数被分开时相应的尺度变化。

图3a和图3b的影响是明确和直接的:随着系数的增加,绕行时间和等待时间都减少了,反映了平台对这些术语的更多强调。相反,图3c的影响并不那么直截了当,但仍然可以解释。随着服务时间(乘车时间)系数的上升,政策倾向于减少绕行和搭车时间,以改善总体乘车时间,这解释了观察到的两种衡量标准都有所下降的原因。在这一背景下,该平台还得益于闲置劳动力能力的增加,使其能够服务更多的订单——特别是旅行距离更长的订单。有观点认为,虽然绕行时间减少,但骑行时间却增加,这表明该平台为完成更长的乘车时间分配了更多的资源,因此,平均乘车时间没有重大变化,而服务率则略有增加。

表5:对于每个目标,相同的堆栈既与目标(i.e,读作w)一起滚动,又与目标(i.e,盲,w=无)分开,在同一注入的奖励下在同一5个测试窗口上滚动。唯一的区别是组合器在推断时是否读懂目标。前两栏对应给定的奖励功能目标;下二对应于由模型从一个简表中写入的自然语言(NL)目标。粗体值突出与给定目标相一致的度量。

皮卡时间 绕行时间 NL 长途旅行 NL 绕行时间 视盲 视盲 视盲 视盲 视盲

服务	0.93±0.04	0.92±0.07	0.93±0.04	0.91±0.05	0.93±0.04	0.89±0.10	0.93±4	0.90±0.05	完成	0.82±0.03	0.82±0.06
	0.82±0.03	0.72±3	0.72±3	0.80±3	0.80±4	等待	1.66±13	1.39±0.21	1.66±0.13	1.80±0.48	1.66±13
	3.37±0.56	1.66±0.13	2.07±0.76	R5.33±0.20	5.27±0.32	5.33±0.20	7.07±0.44	5.33±0.20	5.10±0.24	Detour	0.30±4
	0.300±0.1	1.75±0.19	0.300±0.4	0.16±0.01	Util	0.84±0.14	0.91±0.09	0.84±0.14	0.87±0.11	0.84±0.14	0.89±0.09
	0.84±0.14	0.88±0.09									

在这一部分中,我们探索了政策变化如何响应不同的目标,无论是通过直接的数学奖励功能,还是通过自然语言(NL)描述,结果见表5,其中我们观察到政策对每个目标都作了相应的调整。

对于数学奖励功能(将时间最小化,将绕行时间最小化),我们观察到相应的衡量标准会像预期的那样下降。对于基于NL的目标,我们首先指示政策优先服务长途订单,这通常能带来更高的收入。我们观察到,乘车时间增加了1.74分钟,而绕行时间则增加了1.45分钟,这表明剩下的0.19分钟差异来自政策调整,然而,这一策略不可避免地增加了绕行时间,因为沿途的大部分订单都是短途的。最后,我们以NL目标测试政策,以尽量减少绕行时间。结果显示,这个NL驱动目标实现的性能与数学上指定的对应目标相当,表明LLM自己正确解释自然语言目标。

D.5 公平性

我们研究效率与公平之间的权衡,将公平强度从0(残疾)到1.0不等。表6报告业绩衡量尺度和车辆公平标准差(std,较低)

在所有基线、MARL剂和RideSkill变体中,每个细胞都通过五个相邻的域内情景(大小800、1000和1200;小时17、18和19)进行评估。我们观察到,我们的方法比其他方法更能实现整体上的公平,这可以归因于将与公平相关的技能纳入技能仓库本身。随后,我们注意到,我们提出的弹性公平预算机制在政策中特别有效,没有重定位:增加通用性有助于减少车辆的度量偏差,而不会对总体性能产生重大影响。然而,当重定位者活跃时,这一机制就不那么有效,因为即使没有公平预算机制,重定位者也已经直接改善了公平度量。这是因为重定位器通过将闲置的车辆(通常是收入较低的车辆)迁移到高需求地区来平衡供需。尽管如此,我们认为这一机制仍然有意义,因为有些平台可能无法完全控制车辆的移动,而且重定位可能并非总能得到支持。

表6:效率与公平之间的权衡,在五种情况下均匀。左侧:性能衡量标准。右侧:车辆公平标准,较低者更公平。

性能 Fairnessstd 配置 Reward Service Complete Orders served Dist. W-Dist. 星洲网. 星洲网.

最近的距离 6,779±867 0.79±0.068 0.657 0.79±0.068 0.060 2.45±0.17 2.31±0.14 3,916±1,269 2,269 296±654 KM
6,091±659 0.707±0.047 3.40±0.39 2.96±0.28 9,787±1,933 4,649±237 GS 5,330±749 0.644±0.074 0.512±0.064 3.21±0.27
2.67±0.21 10,787±1,110 5055±342 REDA 7,57±1,090.07 0.062 0.662 0.706 2.00±4 3 333/705 2,601±425 MFRL
7,365±1,157 0.823/0.086 0.678/ 6 0.678/700.079 2.060000.08 1.90±8 1.90±6 1,1 1657±279 [PH] 7,274±1,002
0.807±0.073 0.662±0.066 2.15±0.05 1.99±0.05 2,500±417 2,281±350

RideSkill (w/o reps) + fairness budget

$\rho=0$	9,328±733	0.850±0.059	0.762±0.049	1.84±0.10	1.83±0.09	2,114±644	1,904±500
$\rho=0.25$	9,361±723	0.854±0.055	0.763±0.048	1.64±0.06	1.64±0.05	1,904±369	1,856±346
$\rho=0.50$	9,380±704	0.857±0.050	0.766±0.044	1.67±0.02	1.67±0.00	1,815±310	1,799±298
$\rho=0.75$	9,368±690	0.859±0.048	0.765±0.042	1.69±0.04	1.70±0.03	1,766±262	1,761±255
$\rho=1.0$	9,363±677	0.859±0.046	0.765±0.041	1.75±0.04	1.76±0.03	1,710±239	1,715±238

RideSkill (full stack)

$\rho=0$	9,470±776	0.859±0.061	0.770±0.051	1.66±0.06	1.67±0.05	957±103	949±96
$\rho=0.25$	9,412±769	0.857±0.060	0.767±0.050	1.63±0.05	1.64±0.03	1,057±109	1,063±107
$\rho=0.50$	9,371±754	0.854±0.058	0.764±0.048	1.72±0.05	1.74±0.04	1,100±111	1,113±109
$\rho=0.75$	9,366±746	0.853±0.057	0.764±0.047	1.73±0.08	1.74±0.06	1,108±106	1,119±107
$\rho=1.0$	9,306±752	0.850±0.057	0.763±0.047	1.85±0.09	1.86±0.06	1,119±129	1,134±133

D.6 开源 LLM 上的评估

为了评估RideSkill的训练管道是否超越专有型号,我们用GLM-5.1(Zeng等人,2026年)作为LLM的操作者应用了完整的三相管,取代了主要实验中使用的Claude Opus 4.8型号.表7从表1中比较了五轴一般化扫描上的两个型号,对应的是相同的三堆:单技能平均值、RideSkill(w/o repos)和RideSkill.结果显示,Claude Opus 4.8在所有情景中都超过了GLM-5.1,这表明我们管道的性能受到基本LLM能力的影响。尽管如此,我们观察到GLM-5.1仍然超过了域外环境的所有基准,这表明了我们框架设计的稳健性,使模型能够成功适应各种情景。

D.7 示例

在本节中,我们举出三个培训阶段中每个阶段的产出为例,对每个文物,我们既显示生成的代码,也显示LLM同时撰写的自然语言卡片——包括它宣称的目标,它执行的机制或策略,以及对预期行为的描述。

D.7.1 技能: near_short_efficiency

目标(LLM):通过抓住任何可行的短程旅行、推动服务率和分配数量,而不是尽量减少每名乘客等候和乘车时间,尽量扩大服务需求。

描述(LIM):一个硬的可容性闸门允许旅行低于一个微小的独奏时间预算,只拒绝最长的承诺。幸存。

```

# OD-region prior: destination re-demand
prior = 0.0
if phi_ep.od_orders:
    o, d2 = order["origin_region"], order["destination_region"]
    if o >= 0 and d2 >= 0:
        flow = float(phi_ep.od_count[o][d2])
        prior = 0.15*min(1.0, flow*float(phi_ep.od_orders)/40.0)
        prior += 0.10*min(1.0, float(phi_ep.od_in[d2])*8.0)
prior += 0.10*min(1.0, float(order.get("waiting_time",0.0))/scale)
return efficiency + soonest + prior

def noop_score(driver_obs, phi_ep, phi_step):
    s = driver_obs["self"]
    scale = float(phi_step.mean_solo_time)
    if scale is None or scale <= 0: scale = float(phi_ep.scale) or 1.0
    dp = phi_step.demand_pressure
    base = 0.30 + 0.50*(1.0 - dp/(1.0+dp))
    # region-aware: high demand density -> lower wait threshold
    r = s["current_region"]
    local = 0.0
    if r >= 0 and len(phi_step.region_supply) > r and len(phi_step.region_demand) > r:
        sup = max(float(phi_step.region_supply[r]), 0.01)
        local = 0.35*min(1.0, max(0.0, float(phi_step.region_demand[r])/sup - 1.0)*0.5)
    return base + local

```

列表 23：冻结技能 near_short_efficiency (阶段 1, 第 3 代)。

D.7.2 组合器：objective_shape_router_r2

战略(LLM):对完成、座椅、长度、体积、绕行和空/空转系数进行独立检测,然后将每个车辆状态引导至采用主要系数的冷冻专家——一个快速保留低视距专家,负责完成奖励、座椅集中专家、长度较长的专家、音量/空转的广泛覆盖——始终保护 限时压车先。

描述(LLM):我区分了三个车辆状态:按截止时间(机载ETA低于0.7的活体独奏时间)总是会得到用于保护承诺的骑手的路线+slack_预算,而不管目标如何;装满的黑车会得到与目标座位和完成食欲加权的集合/顶级技能;闲置/空置汽车会得到与目标主因系数相匹配的专家。我通过对结构上不同的事件进行测试来读取这个目标——一个已完成的对只分配的事件(完成价格)、一个党-2对党-1事件(席位价格)、一个长索洛对短索洛事件(长价)、一个两顺序对一顺序事件(数量价格)和空/空/空标记事件(反常态)——使这些在同一个场景中的同一辆车在不同的w下以不同技能着陆。完成式 w 将闲置的汽车倾斜到松动_预算_gate_排名+near_short_efficiency(可靠快速低调完成);座位/集合式 w 倾斜到自由_选择_patience,双自由_ratio_lastseat_topoff和座位_first_arginal_load;长度驱动式 w 倾斜到收入+gowin_ratio_gate(并让慢置汽车追逐长途票价);需求压力是场景调节器:高压(scarcity)将每个目标拉向覆盖/快速服务,因为没有人可以挑剔,而低压则让一个收入目标维持很长的票价。

```

def skill_scores(driver_obs, phi_ep, phi_step, w):
    self_obs = driver_obs["self"]
    etas = [d["eta"] for d in self_obs.get("assigned_order_details", [])]
    if d.get("eta") is not None:
        min_slack = min(etas) if etas else None
    m = float(phi_step.mean_solo_time)
    if m <= 1e-6: m = float(phi_ep.scale) or 1.0
    pend = driver_obs.get("pending_orders", []) or []

    # deadline-pressed: protect onboard rider regardless of objective
    if min_slack is not None and min_slack <= 0.7*m:
        return {"enroute":1.0, "slack_budget_gate_ranked":0.5,
                "自由_可选_patience":0.3,"服务":0.2}

    # --- objective probing ---
    # Probe w on structurally different events to recover coefficient structure.
    # For example: completed vs not-completed, party-2 vs party-1,
    # long-solo vs short-solo, two-order vs one-order, empty/idle flags.
    # [Full probing logic: ~60 lines omitted for space; see code.]

```

```

# Route idle/loaded drivers to the specialist matching dominant coefficient
loaded = onboard > 0 or committed > 0
if not loaded:
    # idle driver: match to dominant objective coefficient
    if comp_c > 0.2: return {"slack_budget_gate_ranked":1.0, ...}
    if seat_c > 0.2: return {"freeness_option_patience":1.0, ...}
    if len_c > 0.2: return {"near_short_efficiency":0.8, "revenue":0.5, ...}
    return {"near_short_efficiency":1.0, "seat_first_marginal_load":0.6, ...}
else:
    # loaded-with-slack: weight by objective's seating/completion appetite
    return {"freeness_option_patience":seat_c+0.3, ...}

```

列表 24：冻结组合器 objective_shape_router_r2 (阶段 2，第 0 代)。

D.7.3 重定位器：objective_steered_reposition_scorer

目标(LLM):将一辆闲置的汽车送到目标加权、供应网未来需求最大超过空置成本的地区,目标转向完成/集合目标中的空降/座位丰富的地区,转向吞吐量下最近的原始需求。

说明(LLM):每个候选地区按其实际有效需求加上前OD结构打分,按完成行程在w(座位、完工)下的实际价值进行加权,并按确切的旅行时间和现有的免费供应进行折扣。目标偏重于多党等候接车和大量空投入的地区,因此,目标明显与目标一致。公平性重现了目标——穷困的(被催动的)汽车被推向满足任何需求,而富裕的汽车则被劝阻去追赶热点,被激起对手将获胜。它只会无物胜过停车时返回当前地区。

```

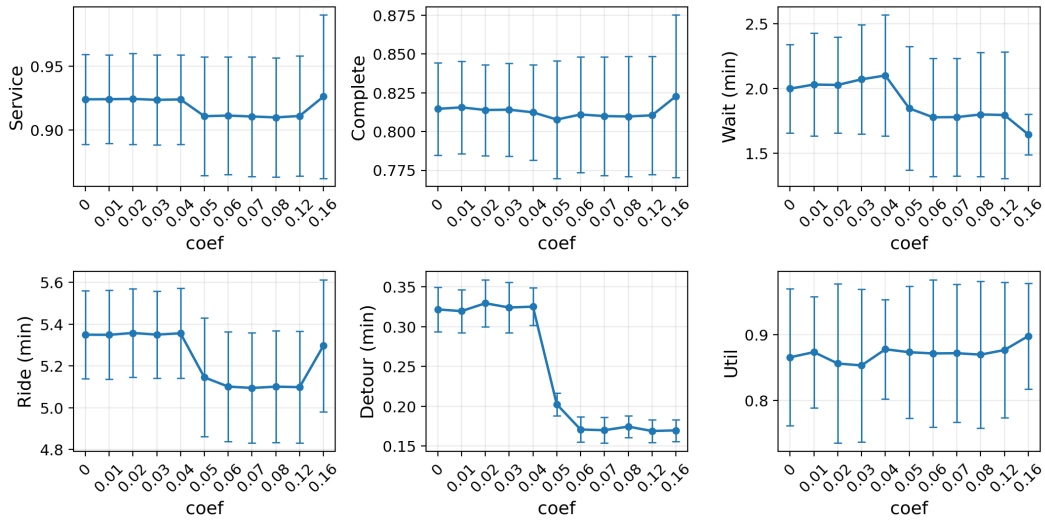
def reposition_scores(driver_obs, phi_ep, phi_step, kappa, w):
    self = driver_obs['self']
    cur = self.get('current_region', -1)
    loc = self['location']
    dist = phi_ep.dist
    scale = phi_step.mean_solo_time
    if scale is None or scale <= 1e-6: scale = phi_ep.scale

    # Probe w to extract trip/seat/empty-cruise values
    trip_val, seat_val, empty_cost = 1.0, 0.0, 0.2
    if w is not None:
        base = {'assigned_orders':[], 'assigned_party_sizes':{}, 'completed_orders':[], ...}
        b0 = float(w(base))
        served = dict(base, assigned_orders=[1], assigned_party_sizes={1:1},
                      completed_orders=[1], ...)
        trip_val = float(w(served)) - b0
        served2 = dict(served, assigned_party_sizes={1:2})
        seat_val = max(0.0, float(w(served2)) - trip_val)
        empty = dict(base, is_empty_move=True, distance_moved=scale)
        empty_cost = b0 - float(w(empty))

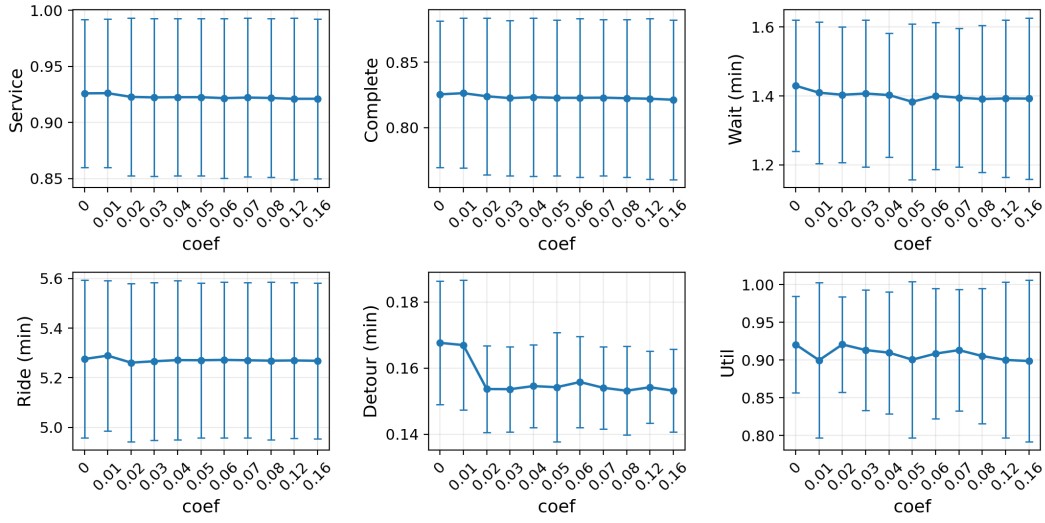
    scores = {}
    for r in range(len(rpts)):
        if r == cur: scores[r] = 0.0; continue
        d = dist(loc, rpts[r])
        cruise_time = max(d / scale, 0.01)
        demand = kappa.eff_demand[r]
        supply = kappa.supply[r]
        net_demand = max(0.0, demand - supply)
        score = (net_demand + seat_val*2.0) * trip_val / (cruise_time + 0.5)
        if score <= empty_cost: score = 0.0 # not worth cruising
        scores[r] = score
    return scores

```

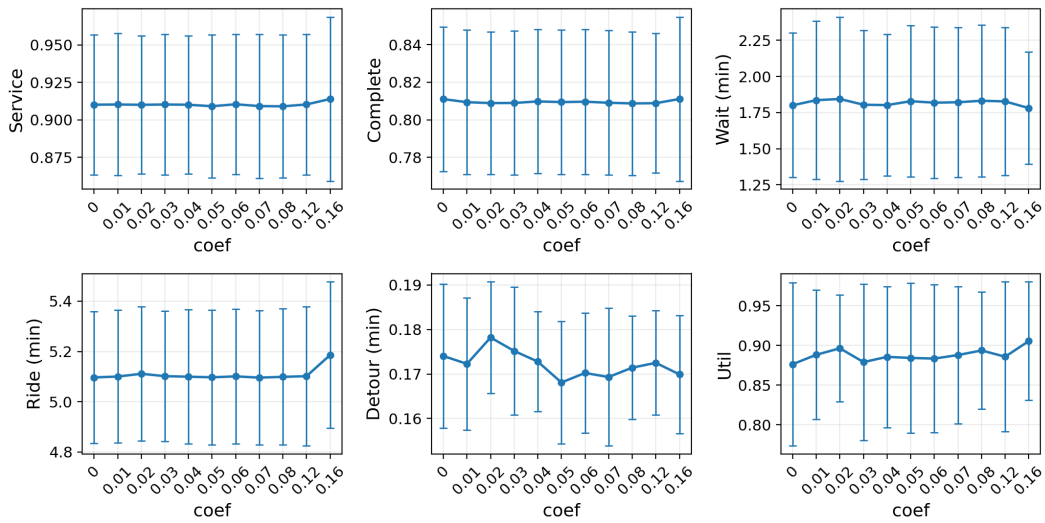
列表 25：冻结重定位器 objective_steered_reposition_scorer (阶段 3，第 5 代)。



(a) 绕道系数扫描



(b) 接驳时间系数扫描



(c) 服务时间扫描

图 3：在不同绕行、接驳时间和服务时间系数下的六个评估指标。